

CHAPTER 2

Database Requirements and ER Modeling

Introduction

The first and most critical step in the process of developing a database is determining the **database requirements** and creating the conceptual database model that represents and visualizes those requirements.

Database requirements are statements that define the metadata (i.e., the details and constraints of the data) for the database that is being developed. Requirements can be derived from interviews with business members, business documentation, business policies, or a combination of these and other sources.

Properly collected requirements should clearly state what the future database will keep track of and in what fashion. **Entity-relationship (ER) modeling**, a widely used conceptual database modeling method, is a technique that enables the structuring and organizing of the requirements collection process and provides a way to graphically represent the requirements.

In this chapter, we will explain how to properly collect database requirements and visually represent them using the ER modeling technique.

Basic ER Modeling Constructs

The result of ER modeling is an **ER diagram (ERD)**, which serves as a blueprint for the database. Two basic ER diagram constructs are the **entity** and the **relationship**. There is no universally adopted ER notation to which all database projects conform. Instead, there is a variety of available ER notations in use. Depending on which ER notation is used, entities and relationships can be represented differently, but they always have the same meaning. In this book, we will use a modified version of the standard Chen ER notation, and all of the constructs will be described in the context of the chosen notation. We have chosen this notation for the following reasons:

- Pedagogical value: easy for novices to learn and use;
- Completeness: All the basic ER concepts are represented.
- Clarity and visibility: All the concepts are graphically represented and every concept is easily distinguished.
- Compatibility with software: This book provides access to the database modeling software tool ERDPlus (available at erdplus.com), which uses the same ER notation.

Once you learn one ER notation, adapting to alternative ER notations is quick and straightforward. At the end of this chapter, we will describe several other ER notations. We will also illustrate how easy and intuitive it is for a developer who is familiar with one ER notation to comprehend and use other ER notations.

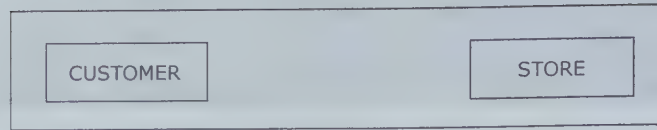


Figure 2.1 Two entities

Entities

Entities are constructs that represent what the database keeps track of. They are the basic building blocks of an ER diagram. Entities¹ represent various real-world notions, such as people, places, objects, events, items, and other concepts. For example, an ER diagram for a retail company may contain entities such as CUSTOMER, STORE, PRODUCT, and SALES TRANSACTION.

In an ER diagram, entities are represented as rectangles, with the name of the entity inside the rectangle. Within one ER diagram, each entity must have a different name. Figure 2.1 shows examples of two entities, CUSTOMER and STORE.

Each depicted entity contains a number of **entity instances** (**entity members**). For example, entity CUSTOMER may contain entity instances such as customer Joe, customer Sue, and customer Pat. Entities themselves are depicted in the ER diagrams, while entity instances *are not*.² However, entity instances will eventually be recorded in the database that will be created based on the ER diagram.

Attributes (Unique and Non-unique)

Each entity in an ER diagram has attributes. An **attribute (of an entity)** describes a characteristic of an entity. While entities are constructs that represent what the database keeps track of, attributes represent the details that will be recorded for each entity instance. For example, for entity CUSTOMER, we may decide to keep track of the following attributes: CustID (customer's identification number), CustName (customer's name), CustBdate (customer's birth date), and CustGender (customer's gender). Figure 2.2 illustrates how attributes are depicted in the ER diagram. Each attribute is represented by an oval containing the attribute name. Within one entity, each attribute name must be different. Every attribute oval is connected by a line to its entity. A **unique attribute** is an attribute whose value is different for each entity instance. Every regular entity must have at least one unique attribute. In an ER diagram, the name of a unique attribute is underlined, as shown in Figure 2.2. The requirements for the database illustrated in Figure 2.2 specify that each customer in the database will have a unique customer identification number (CustID), while we could have more than one customer with the same birth date, name, and/or gender.

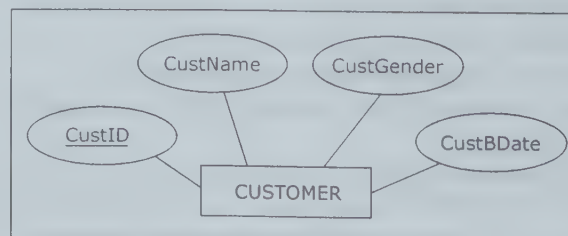


Figure 2.2 An entity with attributes

¹ Entities are also sometimes referred to as "entity types." In this book, we will simply use the term "entity."

² This is because ER diagrams depict the metadata for the future database, and not the data itself.

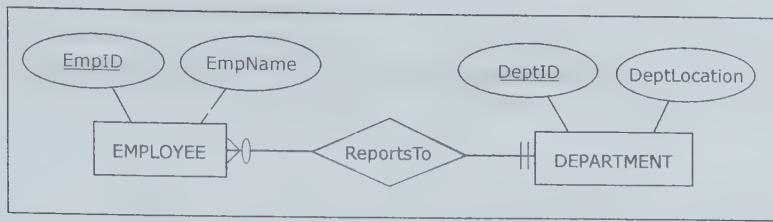


Figure 2.3 A relationship between two entities

Relationships

Within an ER diagram, each entity must be related to at least one other entity via a construct called a relationship. Relationships³ are shown in an ER diagram as a diamond, with a word or phrase naming the relationship inside the diamond. The diamond is connected with lines to the entities that take part in the relationship.

Cardinality Constraints

The lines connecting the relationship to the entities include symbols depicting how many instances of one entity can be associated with instances of another entity. These symbols are known as **cardinality constraints**. Consider the ER diagram shown in Figure 2.3. The diamond with the phrase “ReportsTo” depicts the relationship between EMPLOYEE and DEPARTMENT. Cardinality constraints are marked on an ER diagram as symbols placed next to an entity rectangle on the line connecting the entity rectangle to the relationship diamond.

Each cardinality constraint symbol is composed of two parts:

- **Maximum cardinality**—the part of the cardinality constraint symbol closer to the entity rectangle;
- **Minimum cardinality (participation)**—the part of the cardinality constraint symbol farther away from the entity rectangle.

Maximum cardinality can either be *one* (represented by a straight bar: |) or *many* (represented by a crow’s foot symbol: >).

Participation can be either *optional* (represented by a circular symbol: 0) or *mandatory* (represented by a straight bar: |).

Figure 2.4 shows all four possible cardinality constraints for Entity A in a Relationship B. Cardinality constraint on a particular side of the relationship applies to the entity on the opposite side of the relationship. Note in Figure 2.4 that the name of the right entity (in all four cases) is left blank in order to emphasize that the cardinality constraint depicted on the right applies to the entity on the left.

We will use the example ER diagram containing the relationship ReportsTo (shown in Figure 2.3) to clarify the concept of cardinality constraints. These are the requirements that are used as the basis for the ER diagram shown in Figure 2.3.

- *The database will keep track of employees and departments.*
- *For each employee, we will keep track of his or her name and unique employee ID.*
- *For each department, we will keep track of the unique department ID and the location.*
- *Each employee reports to exactly one department. A department has between zero and many employees reporting to it.*

First, consider the cardinality constraint symbol “||” on the right side of the ReportsTo diamond in Figure 2.3. The *mandatory participation* symbol (left straight bar, in this case) indicates that an employee must report to at least one department. In other words, the minimum cardinality of entity EMPLOYEE in ReportsTo relationship is 1. The maximum cardinality symbol *one* (right straight bar, in this case) indicates that each employee can report to one department at most. In other words, the maximum cardinality of entity EMPLOYEE in ReportsTo relationship is 1. Therefore, those two symbols together indicate that each employee reports to exactly one department.

³ Relationships are also sometimes referred to as “relationship types.” In this book, we will simply use the term “relationship.”

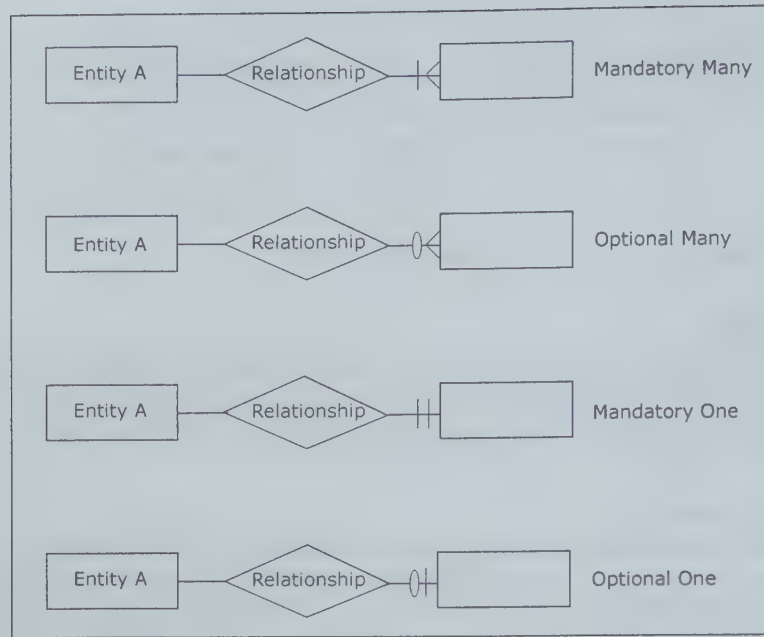


Figure 2.4 Four possible cardinality constraints

Next, consider the cardinality constraint symbol “ $\times$ ” on the left side of the ReportsTo diamond. The *optional participation* symbol (circular symbol) indicates that a department does not have to have any employees reporting to it. In other words, the minimum cardinality of entity DEPARTMENT in the ReportsTo relationship is 0. The maximum cardinality symbol *many* ($\times$) indicates that a department may have many employees reporting to it. In other words, the maximum cardinality of entity DEPARTMENT in the ReportsTo relationship is *many*. Therefore, those two symbols together indicate that a particular department may have many employees reporting to it, but it does not have to have any. In other words, a department has a minimum of 0 employees and a maximum of *many* employees.

Note that the proper way of interpreting a relationship in any ER diagram is to consider it *twice* (once in each direction) by applying the following rule: *rectangle – diamond – cardinality constraint – rectangle*. For example, observe how we interpret the “ReportsTo” relationship:

- One direction: *rectangle* (an employee) – *diamond* (reports to) – *cardinality constraint* (exactly one) – *rectangle* (department);
- Opposite direction: *rectangle* (a department) – *diamond* (has reporting to it) – *cardinality constraint* (between zero and many) – *rectangle* (employees).

To recap the concept of relationship and cardinality constraint, observe a few of the various possible versions of the relationship ReportsTo shown in Figure 2.5 (entity attributes are omitted for brevity).

The following are requirements for each of the versions shown.

Version A:

- Each employee reports to exactly one department. Each department has between zero and many employees reporting to it.

Version B:

- An employee can report to one department or to no departments at all. Each department has between zero and many employees reporting to it.

Version C:

- Each employee reports to exactly one department. A department must have at least one employee reporting to it, but it may have many employees reporting to it.

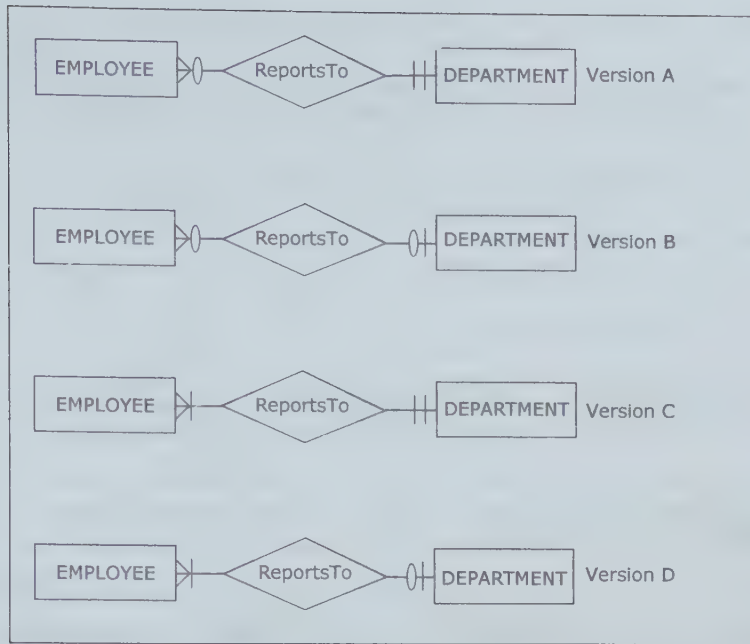


Figure 2.5 Several possible versions of the relationship ReportsTo

Version D:

- *An employee can report to one department or to no departments at all. A department must have at least one employee reporting to it, but it may have many employees reporting to it.*

Types of Relationships (Maximum Cardinality-wise)

The maximum cardinality on either side of the relationship can be either *one* or *many*. Therefore, when considering maximum cardinality without participation, we can classify each relationship between two entities into one of the following three types:

- one-to-one relationship (1:1)
- one-to-many relationship (1:M)
- many-to-many relationship (M:N)⁴

Figure 2.6 illustrates these three types of relationships, maximum cardinality-wise (participation is omitted for clarity, since it does not affect the maximum cardinality).

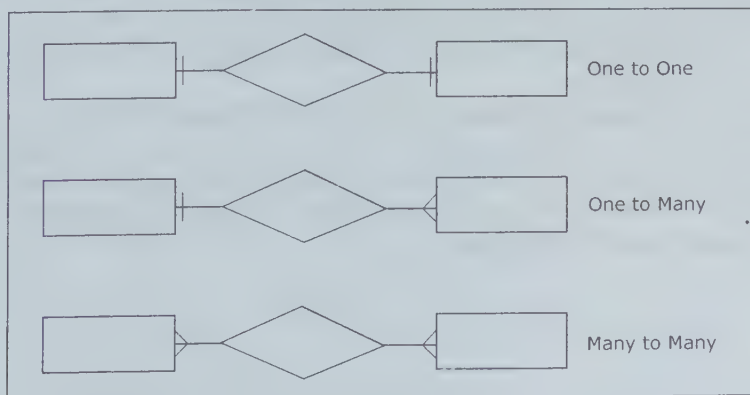


Figure 2.6 Three types of relationships (maximum cardinality-wise)

⁴ M:N is used instead of M:M, to indicate that M and N could be different numbers (both higher than 1), whereas M:M could imply that both “many” cardinalities have to be the exact same number higher than 1.

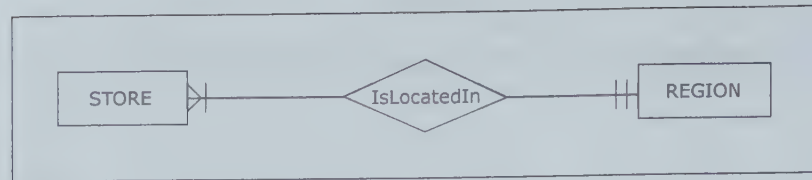


Figure 2.7 A 1:M relationship

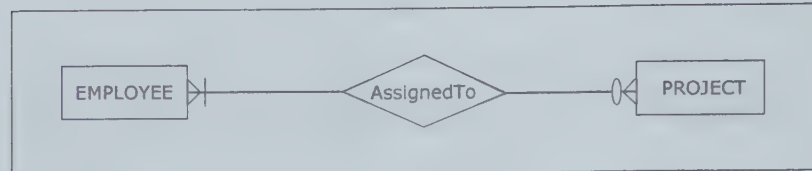


Figure 2.8 An M:N relationship

Figures 2.3 and 2.5 show examples of a 1:M relationship. Another example of a 1:M relationship is shown in Figure 2.7. This relationship reflects the following requirements:

- Each store is located in exactly one region. Each region must have at least one store located in it, but it may have many stores located in it.

Figure 2.8 shows an example of an M:N relationship. This relationship reflects the following requirements:

- An employee is assigned to between zero and many projects. A project must have at least one employee assigned to it, but it may have many employees assigned to it.

Figure 2.9 shows an example of a 1:1 relationship. This relationship reflects the following requirements:

- Each employee is allotted either one vehicle or no vehicles. Each vehicle is allotted to exactly one employee.

Relationships and Relationship Instances

Recall that each entity has its instances. For example, entity EMPLOYEE can have entity instances employee Bob, employee Lisa, employee Maria, and so forth. Entities themselves are depicted in the ER diagrams, while the entity instances are not. However, entity instances are eventually recorded in the database that will be created based on the ER diagram. Similarly, relationships have instances as well. Figure 2.10 illustrates a relationship and an example of its instances.

As illustrated, **relationship instances** occur when an instance of one entity is related to an instance of another entity via a relationship. Just as is the case with entities and their instances, relationships themselves are depicted in the ER diagrams, while the relationship instances are not. However, relationship instances are eventually recorded in the database that is created based on the ER diagram.

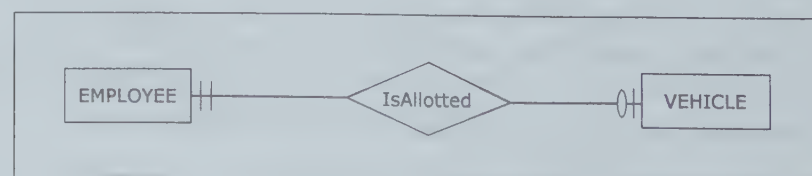


Figure 2.9 A 1:1 relationship

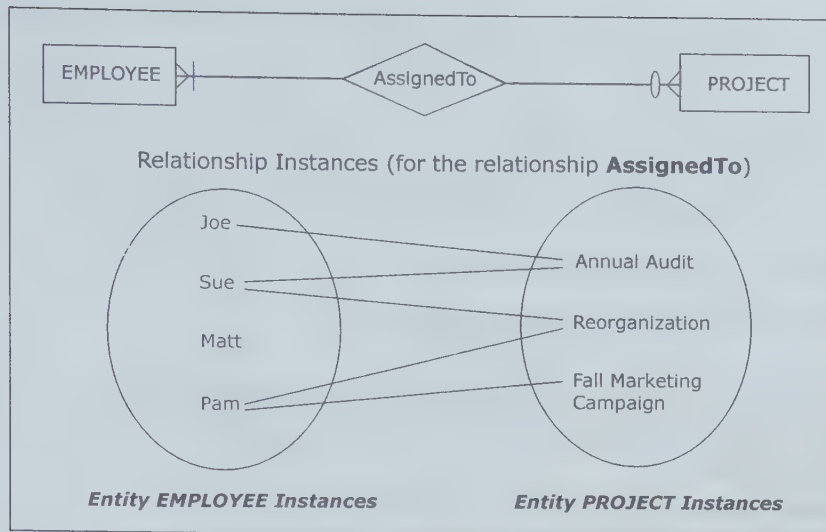


Figure 2.10 A relationship and its instances

Note in Figure 2.10 that entity EMPLOYEE has optional participation in the relationship AssignedTo. That is why, in the bottom part of this figure, it is possible to have some of the actual employees (e.g., Matt) without lines connecting them to actual projects. However, because entity PROJECT has mandatory participation in the relationship AssignedTo, every instance of entity PROJECT is connected to at least one instance of entity EMPLOYEE.

Also note that AssignedTo is an M:N relationship. That is why there are instances of *both* EMPLOYEE and PROJECT entities with multiple lines connecting them to the instances of the other entity.

Relationship Attributes

In some cases, many-to-many relationships can actually have attributes of their own, which are referred to as **relationship attributes**. Figure 2.11 illustrates such a case. The requirements for this ER diagram are as follows:

- The database will keep track of students and campus organizations.
- For each student, we will keep track of his or her unique student ID, and his or her name and gender.
- For each organization, we will keep track of its unique organization ID and the location.
- Each student in the database belongs to at least one organization and can belong to multiple organizations.
- Each organization in the database has at least one student belonging to it and can have multiple students.
- For every instance of a student belonging to an organization, we will record the student's role in the organization (e.g., president, vice president, treasurer, member, etc.).

Note that the last requirement (For every instance of a student belonging to an organization, we will record the student's role in the organization) implies that a student can have several roles, one for each organization he or she belongs to. If we include role as an attribute (or several attributes) of the entity STUDENT, we would not know in which organization the student has what role. Similarly, an organization can have multiple roles, one for each student who belongs to it. If we include role as an attribute (or several attributes) of the entity ORGANIZATION, we would not know which students hold which roles in the organization. Therefore, the only appropriate place to include the Role attribute is in the relationship BelongsTo, as shown in Figure 2.11.

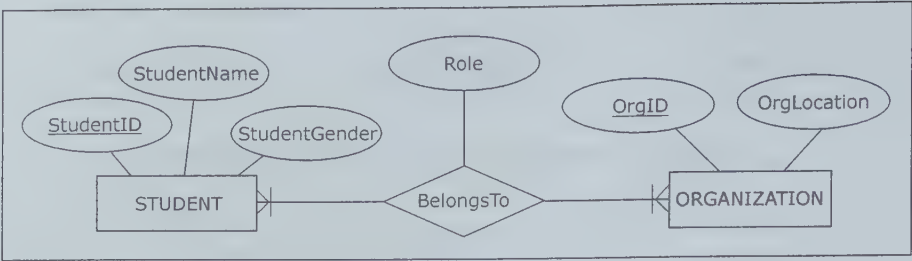


Figure 2.11 A M:N relationship with an attribute

Next, we consider whether 1:1 or 1:M relationships can also have an attribute. To answer that question, consider Figure 2.12, which shows two slightly different ER diagrams based on these requirements:

- The database will keep track of students and colleges.
- For each student, we will keep track of his or her name and unique student ID.
- For each college, we will keep track of its unique name and its location.
- Each student in the database attends exactly one college.
- Each college in the database has multiple students.
- For each student, we keep track of the date he or she started attending his or her college.

The top diagram in Figure 2.12 shows a 1:M relationship with an attribute. The bottom diagram is based on the exact same requirements, but with the student’s college start date as an attribute StudentCollSdate of the entity STUDENT, rather than the attribute DateStarted of the relationship Attends. Since a student attends only one college, his or her start date at that college can be an attribute of the entity STUDENT itself. As illustrated by this example, an attribute of a 1:M relationship can be assigned to the entity whose maximum cardinality in the relationship is 1 (in this case, the entity STUDENT). More generally, a relationship attribute can be assigned to the entity whose maximum cardinality in the relationship is 1, and therefore an attribute of a 1:M relationship or a 1:1 relationship is not necessary. On the other hand, as we illustrated in Figure 2.11, attributes are necessary for some M:N relationships.

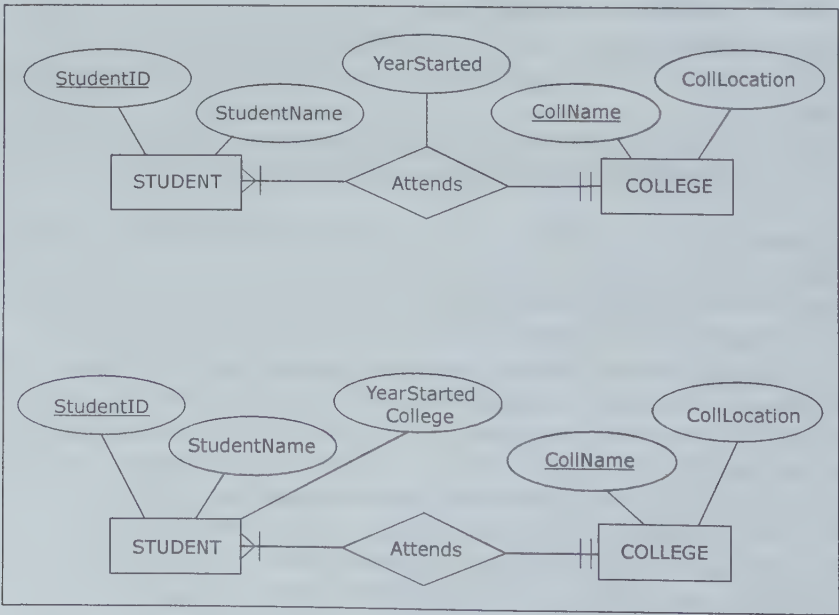


Figure 2.12 A 1:M relationship with and without an attribute

Example: Set of Database Requirements and ER Diagram

The following retail-related example illustrates a collection of database requirements and the subsequent ER diagram.

The sales department of the ZAGI Retail Company has decided to create a database that contains the details of its sales process. After conducting interviews within the company and studying the company documentation, the database team extracted the following requirements for the future database:

ZAGI Retail Company Sales Department Database will capture data about the following:

- For each **product** being sold: a product ID (unique), product name, and price;
- For each **category** of product: category ID (unique) and category name;
- For each **vendor**: vendor ID (unique) and vendor name;
- For each **customer**: customer ID (unique), name, and zip code;
- For each **store**: store ID (unique) and zip code;
- For each **region**: region ID (unique) and region name;
- For each **sales transaction**: transaction ID (unique) and date of transaction;
- Each product is supplied by exactly one vendor. Each vendor supplies one or more products.
- Each product belongs to exactly one category. Each category contains one or more products.
- Each store is located in exactly one region. Each region contains one or more stores.
- Each sales transaction occurs in one store. Each store has one or more transactions occurring at it.
- Each sales transaction involves exactly one customer. Each customer can be involved in one or more sales transactions.
- Each product is included in one or more sales transactions. Each sales transaction includes one or more products.
- For each instance of a product included in a sales transaction, the quantity of that product included (sold) in that transaction is recorded.

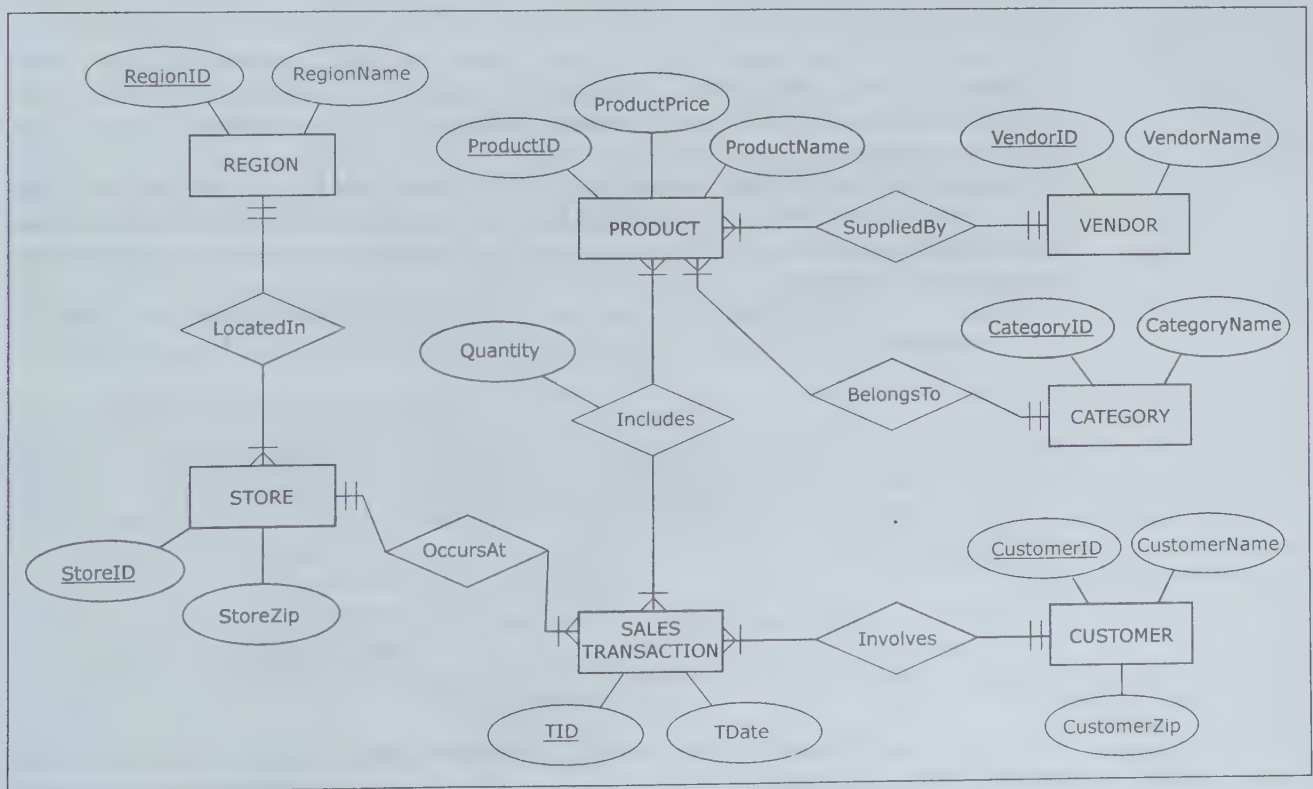


Figure 2.13 An ER diagram example: ZAGI Retail Company Sales Department database

The ER diagram based on these requirements is shown in Figure 2.13. Note that the knowledge of ER notation helps database requirements collectors gather structured and useful requirements for the future database. In other words, knowing that an ER diagram has to be developed upon the completion of the database requirements process helps database requirements collectors focus on asking the right questions. Examples of such questions (posed during the requirements collection process for the diagram shown in Figure 2.13) and answers (received during the same process) are listed here:

- *What in particular would you like to record for each customer?* (Customer's ID, name, and zip code)
- *Is customer ID value unique for each customer?* (Yes)
- *Is customer name value unique for each customer?* (No)
- *Does each product belong to one or multiple categories?* (One)
- *Does each product come from only one vendor?* (Yes)
- *Would you ever record a vendor who does not supply any products?* (No)
- *Do you keep track of customers who did not buy anything yet (i.e., customers that were not involved in any sales transactions yet)?* (No)

Composite Attributes

In addition to the regular attributes shown thus far, ER diagrams can depict several other types of attributes. One additional attribute type is a **composite attribute**—an attribute that is composed of several attributes. Figure 2.14 shows a composite attribute.

In Figure 2.14, attribute CustFullName (customer's full name) is composed of two components: attribute CustFName (customer's first name) and attribute CustLName (customer's last name). The purpose of a composite attribute is to indicate a situation in which a collection of attributes has an additional meaning, besides the individual meanings of each attribute. In Figure 2.14, the depicted entity CUSTOMER has *five* attributes: CustID (depicting customer's unique identification number), CustGender (depicting customer's gender), CustBdate (depicting customer's birth date), CustFName (depicting customer's first name), and CustLName (depicting customer's last name). CustFullName is not an additional attribute of the entity CUSTOMER. Instead, CustFullName simply states that when CustFName and CustLName are combined together, that combination results in the customer's full name.

Figure 2.15 shows another example of a composite attribute. The depicted entity STORE has a total of six attributes, each of which has its own meaning. When the five attributes Street, StreetNumber, City, State, and Zip are considered together they have an additional meaning: Store Address.

Figure 2.16 shows an example of an entity BOUTIQUECLIENT, which has a total of seven attributes. This exclusive tailor-made men's clothing boutique keeps track of the

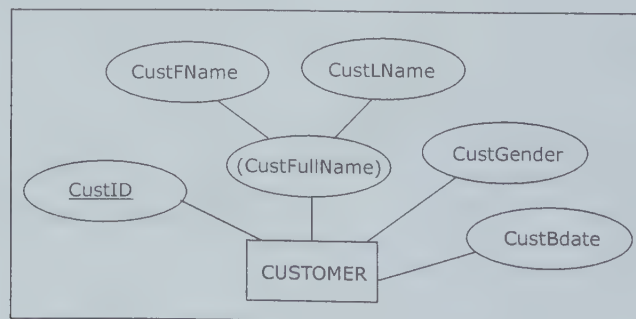


Figure 2.14 An entity with a composite attribute

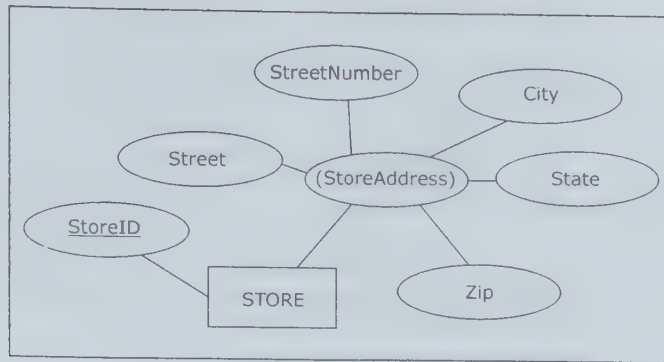


Figure 2.15 Another entity with a composite attribute

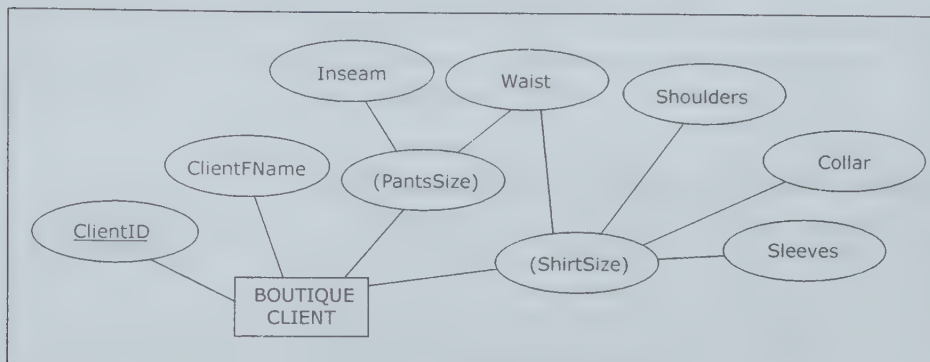


Figure 2.16 Composite attributes sharing components

client's unique identifier and client's first name. Also, the boutique keeps track of the client's five clothing measurements: inseam (length of the leg), waist, sleeves (length of the arm), shoulders (shoulder width), and collar (neck size). This example illustrates how the same simple attribute can be used as a component of more than one composite attribute. In this case, waist is used by two different composite attributes. The combination of inseam and waist is used for the pants size, while the combination of waist, shoulders, collar, and sleeves is used for the shirt size.

Composite Unique Attributes

Figure 2.17 illustrates the case of an entity that has a **composite unique attribute**. In entity **CLASSROOM** shown in Figure 2.17, none of the three attributes (Building, Room Number, and Number of Seats) are unique. The requirements for this entity indicate the following:

- There can be multiple classrooms in the same building (e.g., multiple classrooms in Building A).
- There can be multiple classrooms with the same room number (e.g., Room 111 in Building A versus Room 111 in Building B).
- There can be multiple classrooms with the same number of seats (e.g., multiple classrooms with 40 seats).

Since none of the three attributes are unique, this entity does not have a regular (single-component) attribute that can serve as its identifier. However, the combination of two attributes, Building and Room Number, is unique (there can be only one classroom in the

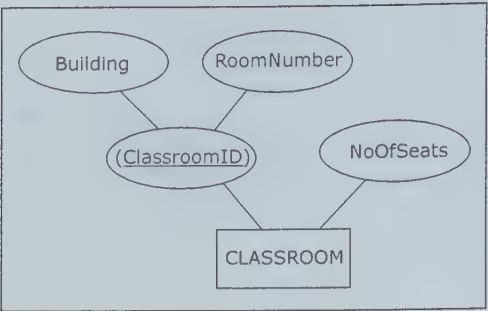


Figure 2.17 An entity with a composite unique attribute

entire resulting database with a particular combination of values for Building and Room Number), and therefore a composite attribute combining these two attributes is unique.

Note that the unique composite attribute ClassroomID ensures that the entity CLASSROOM is in compliance with the rule that each entity must have at least one unique attribute.

Multiple Unique Attributes (Candidate Keys)

Figure 2.18 illustrates a case in which an entity has more than one unique attribute. In such cases, each unique attribute is also called a **candidate key**. The term “candidate key” comes from the fact that a candidate key is a “candidate” to be chosen as the primary identifier (also known as a primary key) when implementing the resulting database. In other words, one of the candidate keys will be chosen later as the primary key for the table corresponding to the entity that contains the candidate keys. Primary keys will be discussed in Chapter 3.

In Figure 2.18, attributes Employee ID and SSN uniquely identify individual instances of the EMPLOYEE entity. Therefore, both Employee ID and SSN are candidate keys. Note that both of these unique attributes are underlined. Attribute Salary is not unique and is therefore not underlined.

An entity can have regular (single-component) and composite candidate keys at the same time. The example in Figure 2.19 shows entity VEHICLE whose attribute VIN (vehicle identification number) is unique, as is the composite attribute License Plate, which is composed of attributes State and License Plate Number.

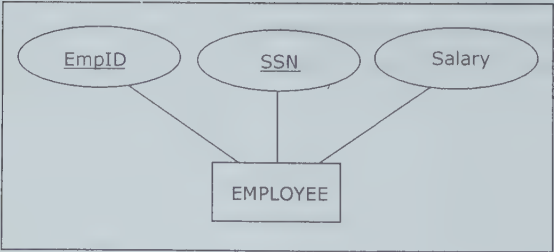


Figure 2.18 An entity with multiple unique attributes

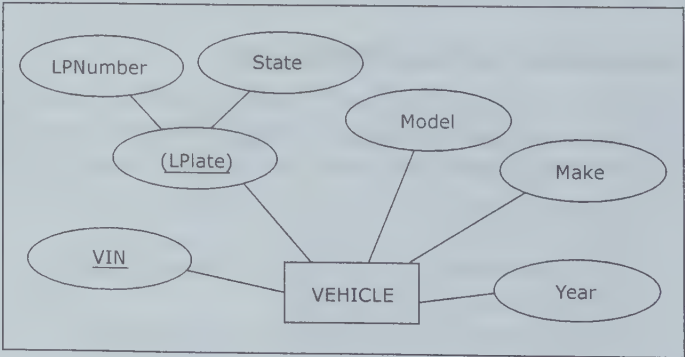


Figure 2.19 An entity with a regular and composite candidate key

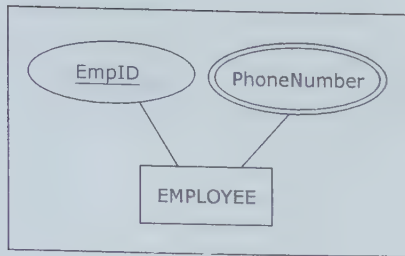


Figure 2.20 A multivalued attribute

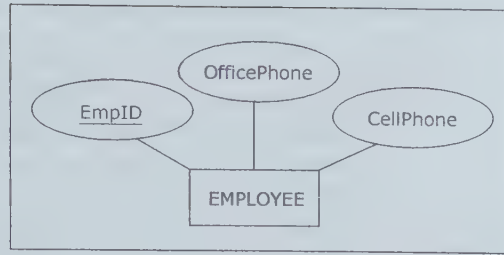


Figure 2.21 A scenario that does not use multivalued attributes

Multivalued Attributes

A **multivalued attribute** is used in cases where instances of an entity can have multiple values for the same attribute. Figure 2.20 depicts an entity that contains a multivalued attribute. The multivalued attribute is depicted by a double line around the oval. The ER diagram in Figure 2.20 shows that we want to be able to keep track of employees' multiple phone numbers.

The multivalued attribute is used in cases in which there is a variable number of values that can be assigned to the particular attribute of the entity. The example in Figure 2.20 is applicable to a situation in which employees have a varying number of phone numbers of which we want to keep track. In particular, some employees may have two phone numbers, other employees may have more than two phone numbers, and yet other employees may have just one phone number.

If, for every employee, we want to keep track of *exactly* two phone numbers (e.g., office phone number and cell phone number), we would *not* use a multivalued attribute. We would simply use two separate attributes, as shown in Figure 2.21.

Derived Attributes

A **derived attribute** is an attribute whose value will not be permanently stored in a database. Instead, the value of a derived attribute will be calculated from the stored values of other attributes and/or additional available data (such as the current date). Figure 2.22 shows a small example that illustrates a derived attribute in an ER model.

A derived attribute is depicted by a dashed oval line. In this case, attribute OpeningDate is a regular attribute whose value will be stored in the resulting database. Attribute YearsInBusiness is a derived attribute whose value will not be stored in the resulting database. Instead, the value for YearsInBusiness will be derived from the value of the store's OpeningDate and from the current date. If attribute YearsInBusiness was a regular attribute, its value in the resulting database would be stored and it would have to be reentered manually as time goes by (once a year for each store), or the database would contain inaccurate information. By depicting YearsInBusiness as a derived attribute, we

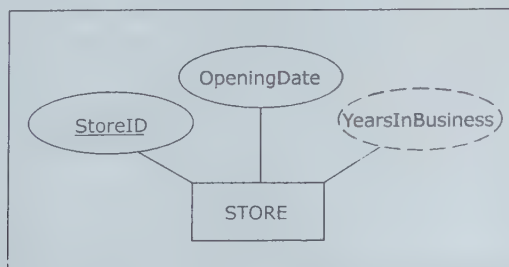


Figure 2.22 A derived attribute example

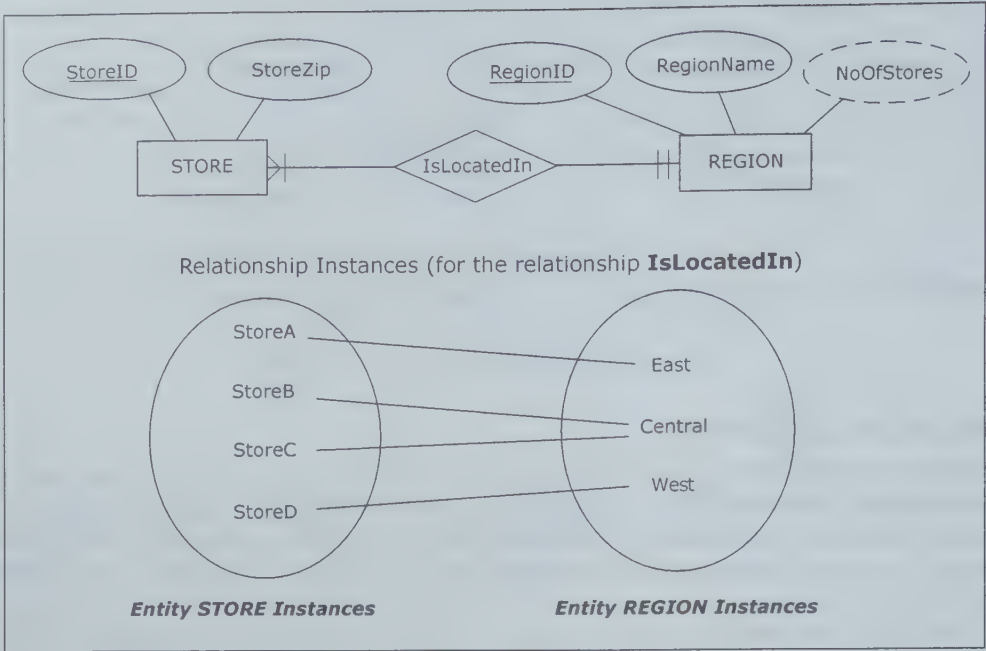


Figure 2.23 Another derived attribute example

assure that the subsequent database will implement this attribute as a formula (Today's Date – OpeningDate, expressed in years) that will always present an accurate value of the YearsInBusiness attribute to the users of the database.

Figure 2.23 shows another example that illustrates a derived attribute. In this case, the values for the derived attribute NoOfStores will be derived by counting how many instances of entity STORE are connected to each instance of the entity REGION via the instances of the relationship IsLocatedIn.

In this example, for regions *East*, *Central*, and *West*, the values of the NoOfStores attribute will be 1, 2, and 1, respectively. If, for example, the entity STORE gets a new instance StoreE, and if this new store is connected with entity REGION instance West via a new instance of the relationship IsLocatedIn, then the value for the derived attribute NoOfStores for the *West* region will automatically increase from 1 to 2.

Optional Attributes

Most of the attributes in an entity will have a value for each entity instance, but some of the attributes may be allowed to not have a value. Such an attribute is called an **optional attribute**. Figure 2.24 illustrates the example of an optional attribute. The example is based on the following requirements:

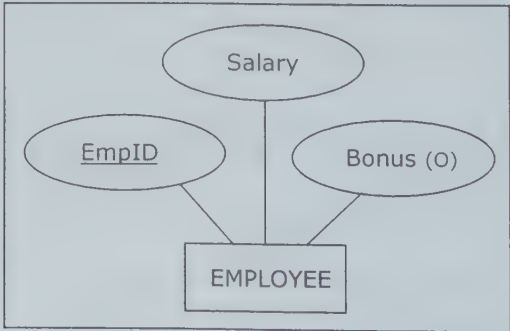


Figure 2.24 An optional attribute

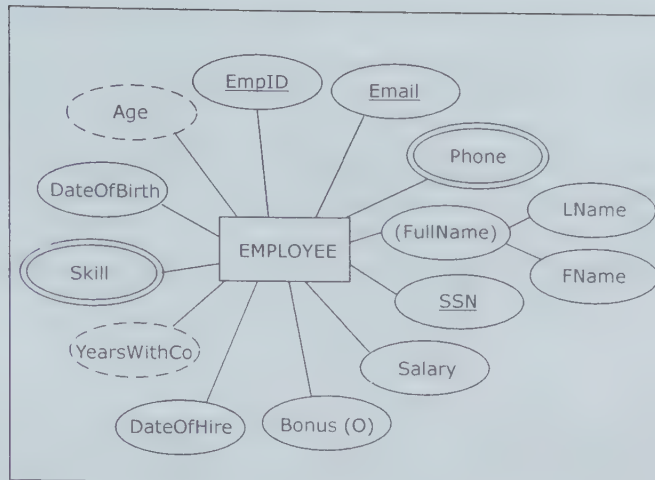


Figure 2.25 An entity with various types of attributes

- For each employee, we keep track of his or her unique employee ID, and his or her salary and annual bonus. Not every employee will have an annual bonus.

The depicted entity has one optional attribute, while the rest of the attributes are required attributes. The optional attribute is depicted by a capital letter O in parentheses at the end of the attribute's name.

Example: Entity Containing Various Types of Attributes

To recap the various types of attributes, Figure 2.25 shows an entity that contains a number of different types of attributes. This entity reflects the following requirements:

- The database will keep track of employees.
- For each employee, we will keep track of the following: an employee's unique ID, unique e-mail, first name, last name (first and last name can be combined into a full name), one or more phone numbers, date of birth, age (which will be calculated from the date of birth and the current date), salary, bonus (optional), one or more skills, unique Social Security number, the date the employee was first hired, and the number of years he or she has worked in the company (which will be calculated from the date of hire and the current date).

Exact Minimum and Maximum Cardinality in Relationships

In some cases, the **exact minimum and/or maximum cardinality** in relationships is known in advance. In our notation, we use pairs of numbers in parentheses placed on the relationship lines to depict the exact values for minimum and maximum cardinality. The first number in the pair, the number next to the open parenthesis, indicates minimum cardinality. The second number, the number next to the closed parenthesis, indicates maximum cardinality. These pairs of numbers can be placed on either side or on both sides of the relationship, depending on which exact minimum and maximum cardinalities are known in advance.

For example, the requirements for the relationship *EnrollsIn*, shown Figure 2.26, state the following:

- Each student must enroll in at least 2 and at most 6 classes, while a class must have enrolled in it at least five students and at most 40 students.

Specific maximum and minimum cardinalities are depicted by the pairs of numbers between the diamond and the cardinality constraint symbols.

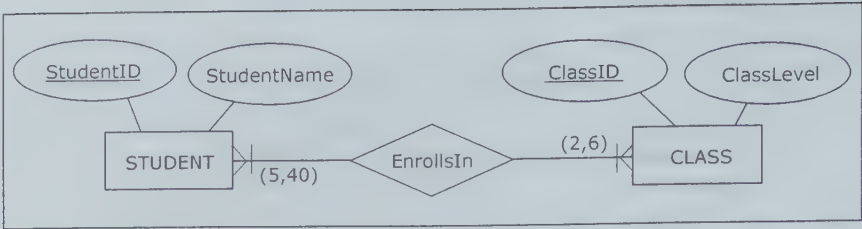


Figure 2.26 A relationship with specific minimum and maximum cardinalities

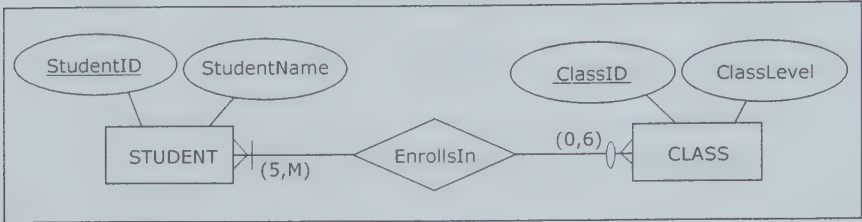


Figure 2.27 A relationship with a mixture of specific and non-specific cardinalities

In this notation, when we are depicting minimum–maximum cardinality as a pair in parentheses, we must show both values (min and max), even if one of them is non-specific. For example, the requirements for the relationship *EnrollsIn*, shown in Figure 2.27, state:

- Each student enrolls in at most 6 classes but does not have to enroll in any. A class must have at least 5 students enrolled in it but can have more than 5 students.

In this case, the optional participation of the entity *STUDENT* in the relationship *EnrollsIn* is depicted twice: once by the optional participation symbol for minimum cardinality, and again by the number 0 as the minimum cardinality in the number pair that appears between the relationship and the cardinality constraint symbol. Also, the multiple (but without a specific number) maximum cardinality of the entity *CLASS* is depicted twice: once by the crow’s foot symbol for maximum cardinality, and again by the letter M as the maximum cardinality in the number pair that appears between the cardinality symbol and the relationship.

Unary Relationships and Relationship Roles

The **degree of a relationship** indicates how many entities are involved in a relationship. So far, every relationship we have examined involves two entities. A relationship between two entities is called a **binary relationship** or a relationship of degree 2 (because it involves two entities). Even though the vast majority of relationships in business-related ER diagrams are binary relationships, there are relationships of other degrees that can occur as well. A relationship of degree 1, also known as a **unary relationship (recursive relationship)**, occurs when an entity is involved in a relationship with itself. Figure 2.28 shows examples of three unary relationships:

1:M, M:N, and 1:1.

The requirements for Figure 2.28, Example A, are:

- A client refers between zero and many clients. Each client is either referred by one other client or is not referred at all.

The requirements for Figure 2.28, Example B, are:

- An employee advises between zero and many employees. An employee is advised by between zero and many employees.

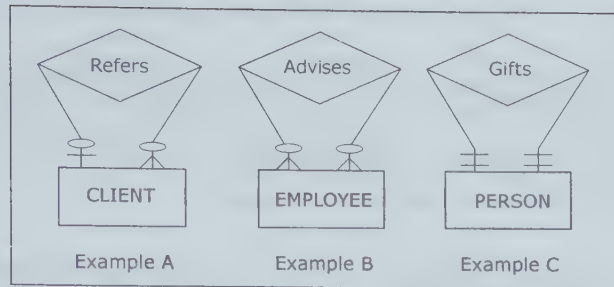


Figure 2.28 Unary relationship examples

The requirements for Figure 2.28, Example C, are:

- In a database for a gift-giving event (Secret Santa), each person gives a gift to exactly one person, and each person receives a gift from exactly one person.

A **relationship role** represents additional syntax that can be used in ER diagrams (at the discretion of a data modeler) to clarify the role of each entity in a relationship. Relationship roles can be used in relationships of any degree, but their usefulness is most apparent when they are used in unary relationships. Relationship roles are represented by the text placed next to the relationship lines. Figure 2.29 shows unary relationships with specified relationship roles.

Diagrams in Figure 2.29 can now be additionally interpreted as follows.

Figure 2.29, Example A:

- A client can be a referring party and as such can refer any number of clients. A client does not have to be a referring party.
- A client can be a referred party and as such can be referred by only one client. A client does not have to be a referred party.

Figure 2.29, Example B:

- An employee can be an advisor for between zero and many employees.
- An employee can be an advisee of between zero and many employees.

Figure 2.29, Example C:

- A person must be a giver of a gift for exactly one person.
- A person must be a recipient of a gift from exactly one person.

Figure 2.30 shows an example of relationship roles in a binary relationship.

As we already mentioned, usage of relationship roles is at the discretion of the ER diagram modeler. Relationship roles can certainly provide clarification in some cases. However, if overused they can also represent unnecessary clutter. For example, in Figure 2.30 a

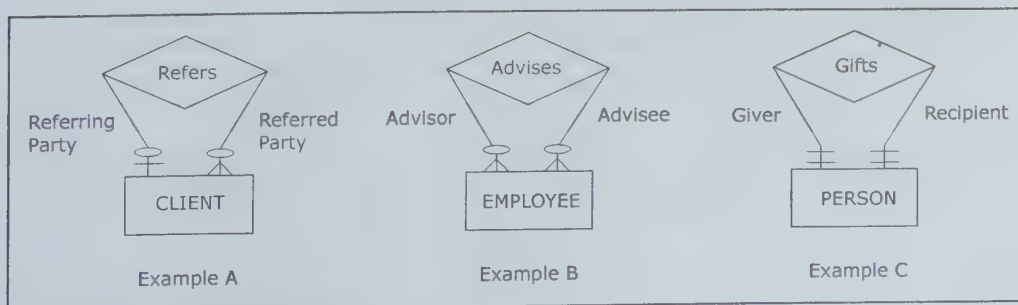


Figure 2.29 Unary relationships with role names

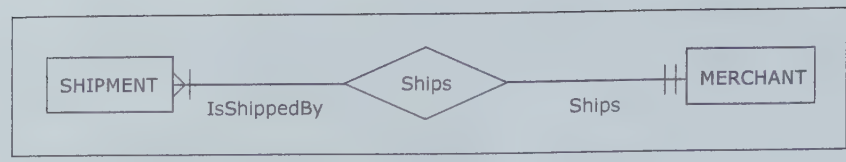


Figure 2.30 A binary relationship with role names

competent observer would most likely not need the explicitly stated relationship roles to interpret the “Ships” relationship. Therefore, the same ER diagram without the relationship roles would be just as informative, while containing less verbiage.

Multiple Relationships Between Same Entities

It is not unusual for the same entities in an ER diagram to be related via more than one relationship. Figure 2.31 shows an example of such a scenario. The example is based on the following requirements.

- A shipping company wants to create a database to keep track of its employees and packages.
- Each package is picked up by exactly one employee.
- Each employee picks up multiple packages.
- Each package is delivered by exactly one employee.
- Each employee delivers multiple packages.

Weak Entity

Recall that each regular entity must have at least one unique attribute. A **weak entity** is a construct in an ER diagram used to depict entities that do not have a unique attribute of their own. A weak entity is represented by a double line depicting a frame of its rectangle. Within an ER diagram, each weak entity must be associated with its **owner entity** via an **identifying relationship**. This type of relationship is represented as a double-framed diamond.

Figure 2.32 illustrates an example of a weak entity. The example is based on the following requirements:

- An apartment rental company wants to create a database to keep track of its buildings and apartments.
- For each building, we will keep track of a unique building ID and the number of floors in the building.
- For each apartment, we will keep track of the apartment number and the number of bedrooms in the apartment.

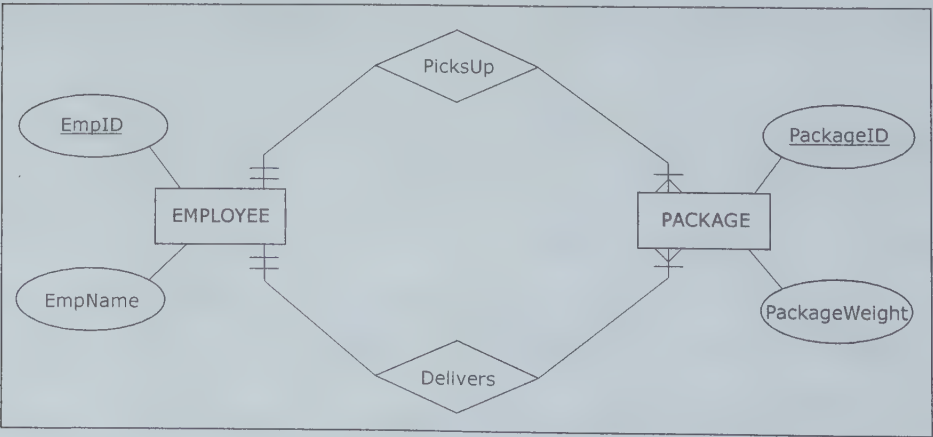


Figure 2.31 Multiple relationships between the same entities

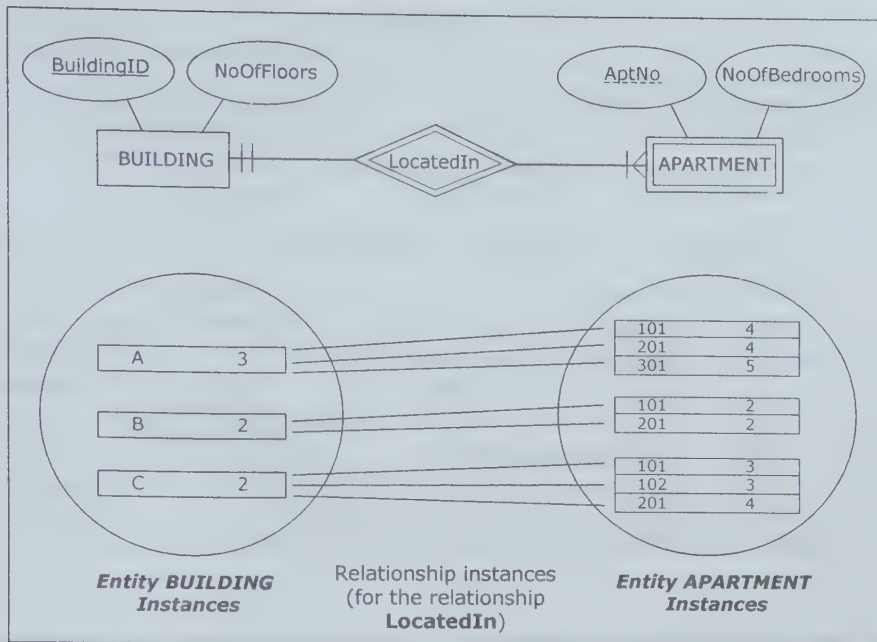


Figure 2.32 A weak entity example with entity instances

- Each building has multiple apartments, and each apartment is located in one building.
- We can have multiple apartments with the same apartment number in our database, but within one building each apartment will have a different apartment number.

As indicated by the requirement, there is no unique attribute for the entity APARTMENT. However, the apartment number attribute is unique within each building. Such an attribute is called a **partial key**, and it is depicted in the ER diagram by a dashed underline. A combination of the partial key and the unique attribute from the owner entity uniquely identifies every instance of the weak entity. For example, instances of the weak entity APARTMENT are uniquely identified by combining the value of their partial key AptNo and the primary key BuildingID of their owner entity BUILDING, as follows:

A 101, A 201, A 301, B 101, B 201, C 101, C 102, C 201.

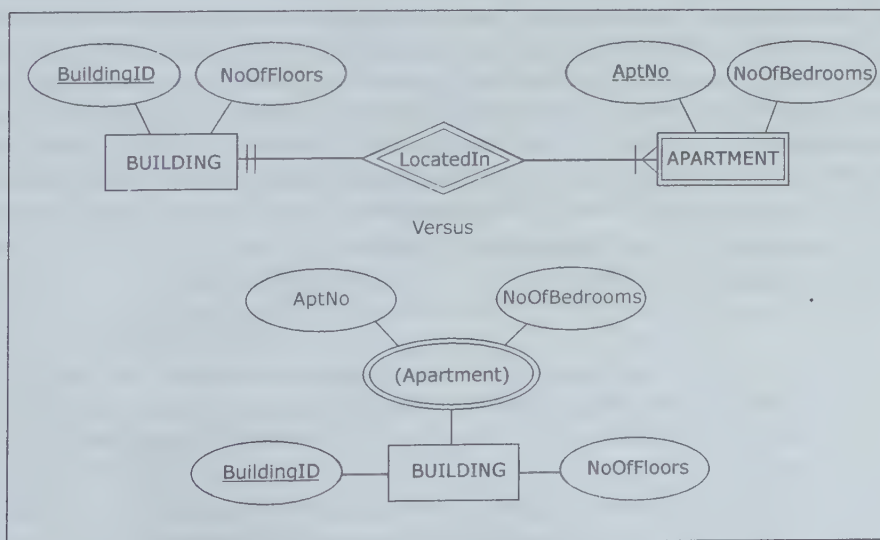


Figure 2.33 A weak entity (top part) versus a multivalued composite attribute (bottom part)

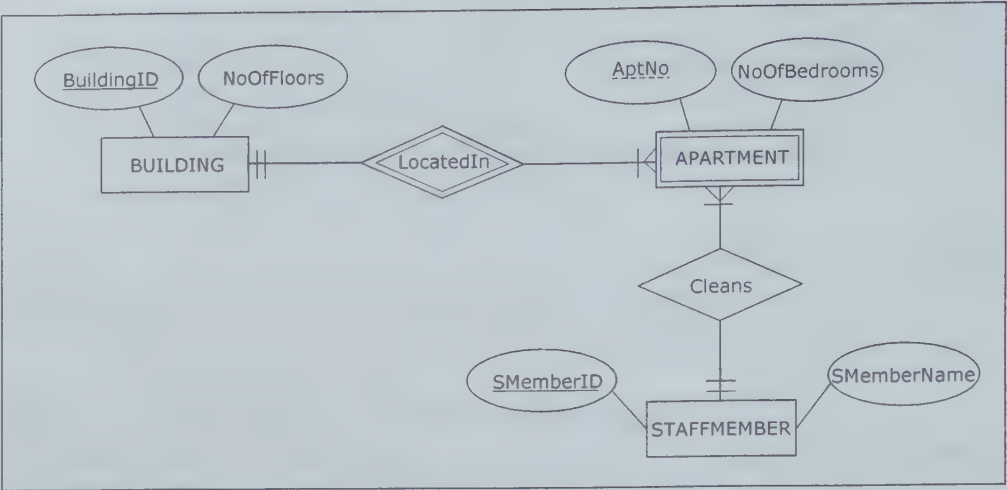


Figure 2.34 A weak entity with an identifying and regular relationship

The concept of a weak entity is actually similar to the concept of a multivalued composite attribute. Figure 2.33 illustrates this point.

Both diagrams in Figure 2.33 represent the same requirements. However, the weak entity notation is capable of depicting a partial key, while there is no such option for the multivalued composite attribute. For example, in the top part of Figure 2.33, we can depict that each apartment number is unique for apartments within one building, while in the bottom part of Figure 2.33 we cannot.

A weak entity can have regular (non-identifying) relationships with other entities. An example of a weak entity engaged in a non-identifying relationship is given in Figure 2.34. If we used a multivalued composite attribute instead of a weak entity, we would not be able to depict such a relationship.

Even though weak entities and multivalued composite attributes are similar at first glance, there are meaningful differences between these two concepts, as we have just illustrated.

Each weak entity always has the mandatory-one cardinality constraint in the identifying relationship with its owner. This ensures that each instance of a weak entity is associated with exactly one instance of its owner. The owner entity, on the other hand, can have a mandatory or optional participation in the identifying relationship, as it is possible to have owner entities with instances that do not have weak entity instances associated with them.

In the majority of cases, an identifying relationship is a 1:M relationship. However, it is possible to have a 1:1 identifying relationship, in which case a partial identifier attribute in the weak entity is not necessary. The example shown in Figure 2.35 illustrates such a case. The example is based on the following requirements:

- *In our database, we will keep track of employees and their spouses.*
- *For each employee, we will keep track of his or her unique ID and name.*
- *For each spouse, we will keep track of his or her name and birthdate.*

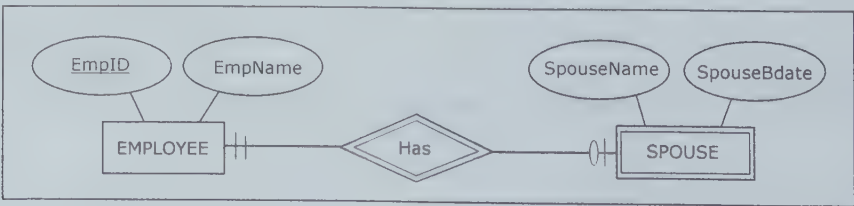


Figure 2.35 A weak entity with a 1:1 identifying relationship

- *An employee either has one spouse or no spouse.*
- *Each spouse is married to exactly one employee.*

Note that the weak entity SPOUSE does not have a partial identifier because each employee can be associated with only one spouse; therefore, we do not need a partial identifier to distinguish one spouse from the other.

Naming Conventions for Entities, Attributes, and Relationships

In the process of ER modeling, a good practice is to adopt certain guidelines for naming entities, relationships, and attributes. In this book, the convention is to use all uppercase letters for naming entities and a combination of uppercase and lowercase letters for naming attributes and relationships.

For naming entities and attributes, a common guideline for keeping ER diagrams as legible and clear as possible is to use singular (rather than plural) nouns. For example, entity names *STUDENT*, *STORE*, and *PROJECT* are better choices than *STUDENTS*, *STORES*, and *PROJECTS*. Similarly, the attribute name *Phone* is a better choice than the attribute name *Phones*, even in the case of a multivalued attribute. Plurality of instances in the case of entities and multivalued attributes is understood by the nature of these constructs, without having to use a word in plural form.

For naming relationships, it is common to use verbs or verb phrases, rather than nouns. For example, *Inspects*, *Manages*, and *BelongsTo* are better choices for names of relationships than *Inspection*, *Management*, and *Belonging*.

When naming entities, attributes, and relationships, it is advisable to be as brief as possible, without being so condensed as to obscure the meaning of the construct. For example, in the case of an ER diagram for a university, the entity name *STUDENT* would be a better choice than *UNIVERSITY STUDENT*, which is unnecessarily wordy, since it is clear that in this context, *STUDENT* refers to university students. Likewise, using *US* as an abbreviation that stands for “university student” may be too cryptic for the average user of the future database based on this ER diagram. That is not to say that in some cases multiple words or abbreviations are not good choices. For example, *SSN* is a perfectly valid choice for a name of an attribute, due to the widespread understanding of the meaning of this abbreviation as a social security number. Also, the multiword phrase *NoOfFloors* is a better choice for naming the attribute that indicates the number of floors for the entity *BUILDING*, than the single word *Floor*, which is too ambiguous, or the abbreviation *NoF*, which is not recognizable by most users.

As we mentioned at the beginning of this chapter, one of the basic rules of ER modeling is that every attribute within the same entity must have a different name. A good style rule of ER modeling is to give all attributes in the entire ER diagram different names. For example, instead of using the word *Name* to represent two different attributes of two different entities *EMPLOYEE* and *CUSTOMER*, we would use two different words, *EmpName* and *CustName*.

The guidelines stated here are not mandatory and are subject to exceptions. However, when applied consistently, these conventions usually result in ER diagrams that are more legible and clearer than ER diagrams created without them.

Multiple ER Diagrams

In an ER diagram, each entity is always connected to all of the other entities, either via a direct relationship or indirectly via other entities and relationships. In other words, within each ER diagram there is a path from one entity to every other entity.

If an ER schema contains entities that are not connected to other entities in the schema, such schema actually represents multiple ER diagrams for multiple separate databases. For example, in Figure 2.36, entities A, B, and C are not connected to entities D, E, F, and G. Therefore, entities A, B, and C are components of one ER diagram, while entities D, E, F, and G are components of another ER diagram.

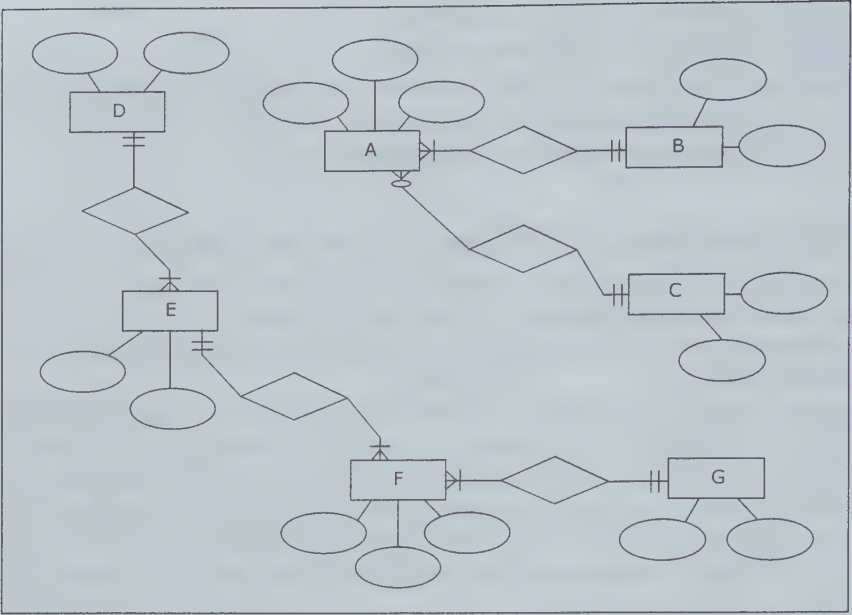


Figure 2.36 A schema with two separate ER diagrams (potentially misleading)

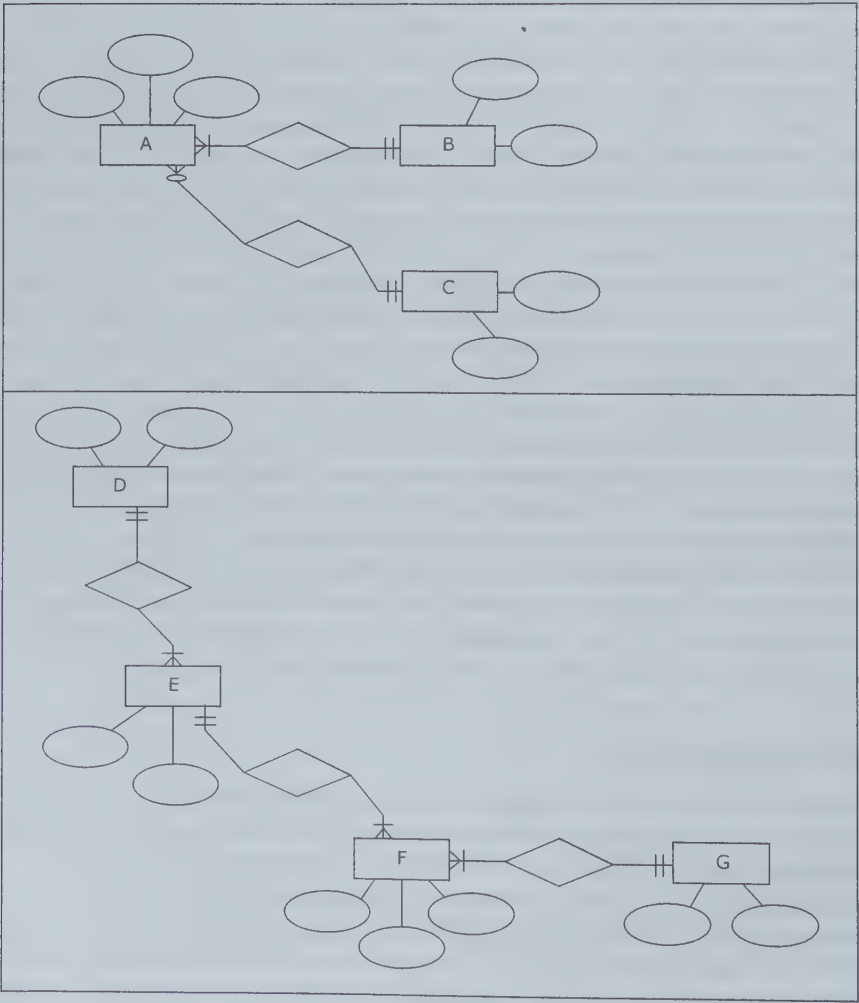


Figure 2.37 Separate ER diagrams in separate schemas

In general, when depicting multiple ER diagrams, each diagram should be *visualized separately*. Instead of two diagrams in one schema, as shown in Figure 2.36, a better choice would be to present each ER diagram individually, as shown in Figure 2.37. This eliminates the possibility of confusion for any viewer that does not notice that the schema contains multiple (non-connected) ER diagrams.

Example: Another Set of Database Requirements and an ER Diagram

In order to recap the introduced ER modeling concepts, we will look at another example of an ER diagram in Figure 2.38, based on the following requirements:

- HAFH (“Home Away from Home”) Realty Company leases apartments to corporate clients. HAFH Realty Company Property Management Database will keep track of HAFH buildings, apartments, corporate clients, building managers, cleaning staff members, and building inspectors.

The property management database for the realty company HAFH will capture data about the following:

- For each building: *BuildingID* (unique) and *BNoOfFloors* (number of floors in the building);
- For each apartment: *AptNo* (partially unique, i.e., unique within a building) and *ANoOfBedrooms* (number of bedrooms in the apartment);
- For each corporate client: *CCID* (unique), *CCName* (unique), *CCLocation*, and *CCIndustry*;
- For each manager: *ManagerID* (unique), *MFullName* (composed of *MFirstName* and *MLastName*), multiple *MPhone* numbers, *MbDate*, *MAge* (derived from *MBdate* and current date), *MSalary*, and *MBonus* (not every manager will have a bonus);

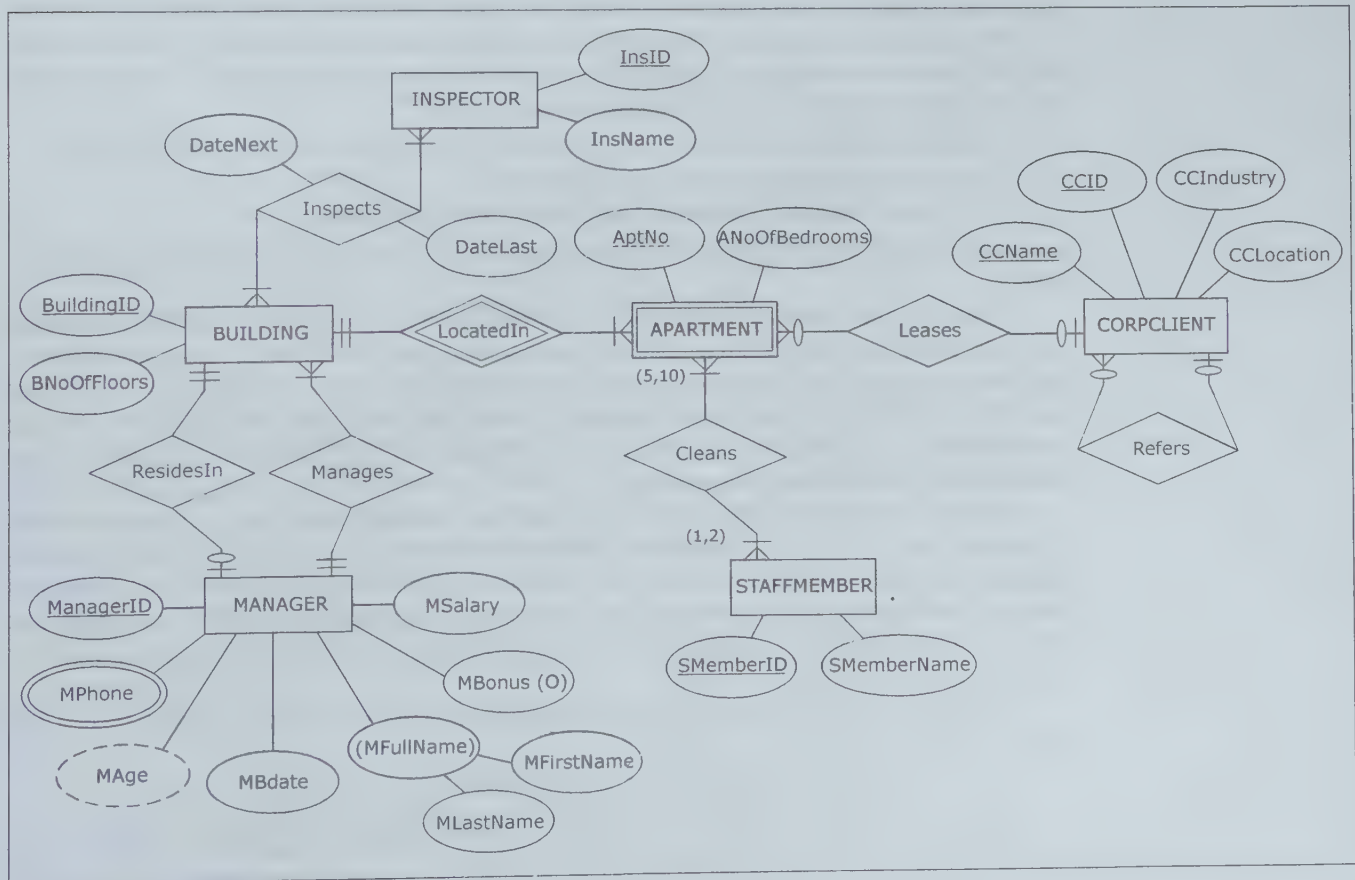


Figure 2.38 Another ER diagram example: HAFH Realty Company property management database

- For each staff member: *SMemberID* (unique) and *SMemberName*;
- For each inspector: *InsID* (unique) and *InsName*;
- Each building has one or more apartments. Each apartment is located in exactly one building.
- Each apartment is either leased to one corporate client or to no one. Each corporate client leases between zero and many apartments.
- Each corporate client refers between zero and many corporate clients. Each corporate client can be referred by one corporate client or is not referred by any.
- Each apartment is cleaned by either one or two staff members. Each staff member cleans between five and ten apartments.
- Each manager manages one or many buildings. Each building is managed by exactly one manager.
- Each manager resides in exactly one building. Each building either has one manager residing in it or no managers residing in it.
- Each inspector inspects one or many buildings. Each building is inspected by one or many inspectors.
- For each building that a particular inspector inspects, the dates of the last inspection by the inspector and of the next future inspection by the inspector are recorded.

Database Requirements and ER Model Usage

ER modeling provides a straightforward technique for collecting, structuring, and visualizing requirements. Working knowledge of ER modeling is crucial, not just for creating ER models based on the requirements, but it is also important during the requirements collection process itself. It helps keep the focus on asking or seeking answers to the right questions in order to establish the relevant facts about entities, attributes, and relationships.

One of the common mistakes that beginners make when engaging in ER modeling for the first time is not recognizing the difference between an entity and the ER diagram itself. For example, consider this set of simple requirements:

- *Company X keeps track of its departments and employees who report to those departments.*
- *Each employee reports to exactly one department.*
- *Each department has between zero and many employees reporting to it.*

An incorrect ER diagram based on these requirements is shown in the top part of Figure 2.39.

Depicting the company as an entity is a mistake. Such an entity would have only one instance (Company X) and it would unnecessarily complicate the ER diagram. Instead, the ER diagram, based on the small set of requirements above, simply has two entities: employee and department, as shown in the bottom part of Figure 2.39. Company X is not represented as an entity. Instead, the entire ER diagram (in this case two entities and their relationship) represents Company X.

Note that if the first bullet in the requirements were different, e.g., “An industry association is keeping track of its member companies, its departments, and its employees,” then the diagram shown at the top of Figure 2.39 would make sense. However, the requirements clearly indicate that this is a database for one company to keep track of its employees and departments; therefore, the bottom diagram in Figure 2.39 is correct for that set of requirements and the top diagram is not.

To further elaborate this point, consider the diagram shown in Figure 2.40. This diagram makes sense for a scenario such as:

- *A non-profit foundation keeps track of the scholarship recipients that it is funding.*
- *For each student (scholarship recipient), it keeps track of his or her unique student ID, name, and the date he or she started college.*
- *For each college attended by its scholarship recipients, the foundation keeps track of the unique college name and college location.*

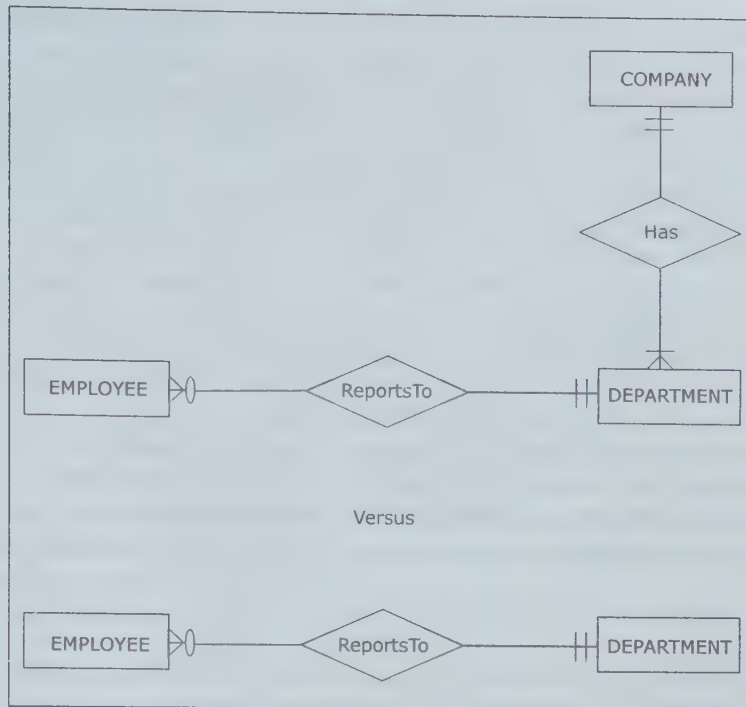


Figure 2.39 An ER diagram incorrectly (top part) and correctly (bottom part) interpreting requirements

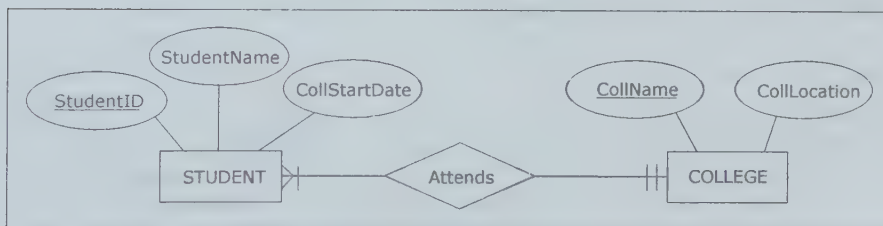


Figure 2.40 An ER diagram containing student and college entities

- Each scholarship recipient attends exactly one college. Each college in the database has at least one of the scholarship recipients attending.

Note that the diagram in Figure 2.40 would not make sense for a database for one individual college keeping track of its students. In that case, entity COLLEGE would be unnecessary, just as the entity COMPANY was unnecessary in Figure 2.39.

Another common mistake made by novices in database requirements collection and ER modeling is not distinguishing between

modeling of the data that is wanted and can be kept track of
versus
 modeling of everything that takes place in an organization.

For example, the following set of requirements fails to make such a distinction:

- Airline X wants to keep track of flights, flight attendants, and passengers.
- Each flight is staffed by many flight attendants. Each flight attendant staffs many flights.
- Each passenger flies on many flights. Each flight has many passengers.
- Each flight attendant serves many passengers. A passenger is served by many attendants.

The top part of Figure 2.41 shows an ER diagram based on these requirements. All of these requirements accurately represent Airline X. The problem is that these requirements do not take into account what is possible and/or useful to keep track of. While the first three lines of requirements are legitimate, the last line is problematic. While it is true that passengers are served by many flight attendants and attendants serve many passengers, it is very unlikely that we would want to actually record every combination of a flight attendant serving a customer that took place. Even if we wanted to record this, we would have to figure out the cost and practicality of doing so. In this example, we would have to have a mechanism that somehow records every occurrence when a flight attendant serves a passenger for the first time (e.g., brings a drink, answers a short question). It is very likely that such a mechanism would be prohibitively expensive, in addition to being intrusive and pointless. Therefore, the last row of requirements should be eliminated, and the database requirements should be listed as follows:

- *Airline X wants to keep track of flights, flight attendants, and passengers.*
- *Each flight is staffed by many flight attendants. Each flight attendant staffs many flights.*
- *Each passenger flies on many flights. Each flight has many passengers.*

Subsequently, the diagram should be constructed as shown in the bottom part of Figure 2.41.

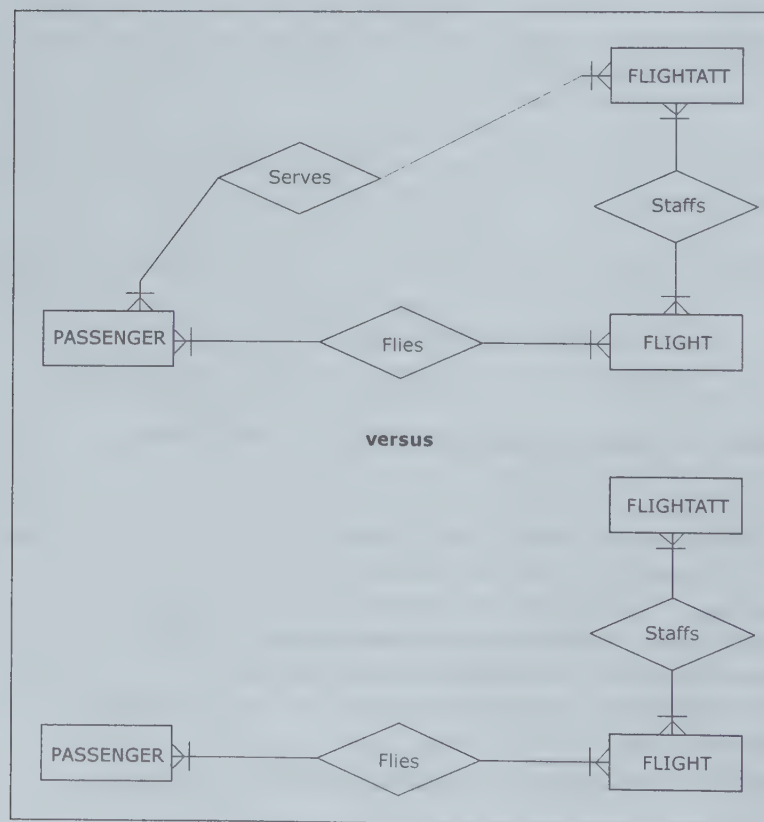


Figure 2.41 An ER diagram based on unfeasible (top part) and proper (bottom part) requirements

Various ER Notations

As we mentioned earlier, there is no universally adopted ER notation to which all database projects conform. Instead, there are a variety of available ER notations currently in use. However, if a designer is familiar with one ER notation, other alternative ER notations are easy to understand and use.

Figure 2.42 illustrates the same exact ER diagram in three different notations. The top diagram in Figure 2.42 illustrates the ER notation that is used in the ER diagrams in this book.

The middle diagram in Figure 2.42 uses a notation in which attribute names are listed in the same rectangle with the entity name, while the relationship is represented as a dashed line connecting the entities with a relationship name embedded in the line. Cardinality constraints are represented by numbers in parentheses. In this notation, cardinality constraints are *reversed* (when compared to the notation used in this book) and the rule for interpretation of relationships is entity – cardinality constraint – relationship – entity. For example, each instance of entity EMPLOYEE participates exactly once in the relationship ReportsTo with the instances of entity DEPARTMENT, while each instance of entity DEPARTMENT can participate between zero and many times in the relationship ReportsTo with the instances of entity EMPLOYEE.

The bottom diagram in Figure 2.42 uses UML (Unified Modeling Language) notation to represent the same ER diagram. This notation uses a similar visualization of entities and relationships as the middle notation, but the rules for interpretation of cardinality constraints are the same as for the notation in the top diagram. Cardinality constraints are represented by two numbers separated by two dots when participation is optional, and by just one number when participation is mandatory.

There are many other ER notations in existence, in addition to the ones shown here. Fortunately, they all depict the same concepts: entities, relationships, and attributes. A designer who is familiar and comfortable with those concepts can quickly adapt to any ER notation.

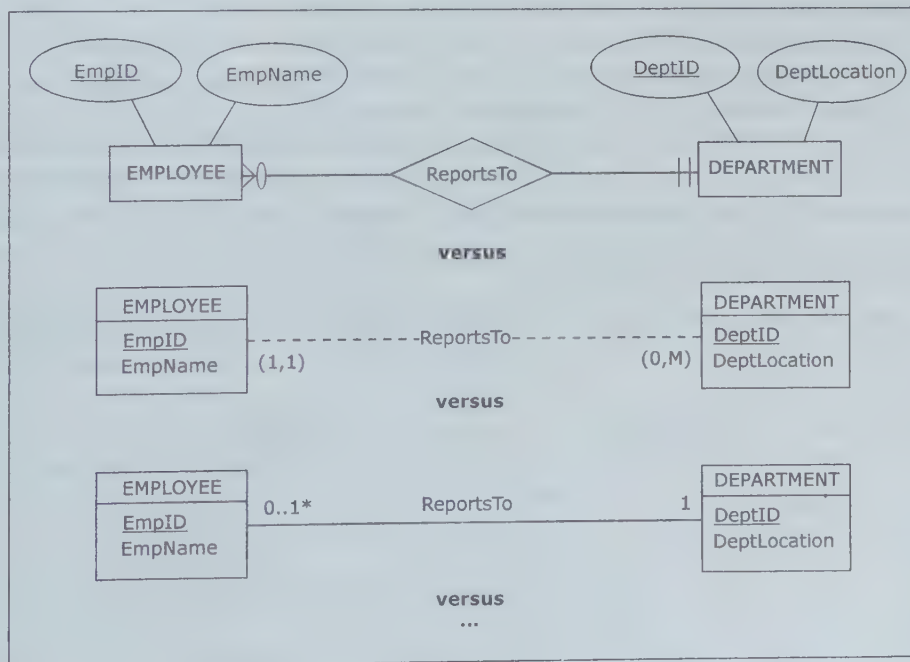


Figure 2.42 Examples of various ER notations

Enhanced ER (EER) Modeling

The term **enhanced ER (EER)** modeling refers to an expanded ER notation that depicts additional concepts beyond standard ER modeling. We give a brief overview of EER in Appendix 1. While the EER extensions of traditional ER concepts have value and use, most common business-related databases can be modeled in large part by the standard ER notation introduced in this chapter. A competent ER modeler can learn and apply EER extensions easily, if and when they are needed.

◆ ◆ ◆

In this chapter we have presented the most fundamental issues related to database requirements and ER modeling. The following notes cover several additional issues related to ER modeling.

A Note About M:N Relationships with Multiple Instances Between the Same Entities

In some cases, M:N relationships can have multiple occurrences between the same instances of involved entities. The following example illustrates such a case. Observe the requirements listed below:

- The database will keep track of employees and proficiency tests.
- For each employee, we will keep track of his or her unique employee ID and name.
- For each proficiency test, we will keep track of the unique proficiency test name and the proficiency test cost.
- Each employee in the database can take between zero and many proficiency tests.
- Each proficiency test in the database was taken by at least one employee and could have been taken by multiple employees.
- For every instance of an employee taking a proficiency test, we will record the proficiency test score and date.

Figure 2.43 shows an ER diagram based on these requirements. Figure 2.44 illustrates some of the possible instances of the relationship shown in Figure 2.43.

If we now add a small but critical addition to the above requirements, we will fundamentally change the nature of the depicted M:N relationship:

- An employee can take the same proficiency test more than once (e.g., if an employee receives a score below a minimum score, he or she has to take the same proficiency test again, until the minimum score is achieved). An employee can take only one proficiency test on the same date.

This addition now allows for multiple instances of a relationship between the same instances of entities. This is illustrated by Figure 2.45, in which employee *Max* takes the same proficiency test *Conflict Mediation* three times.

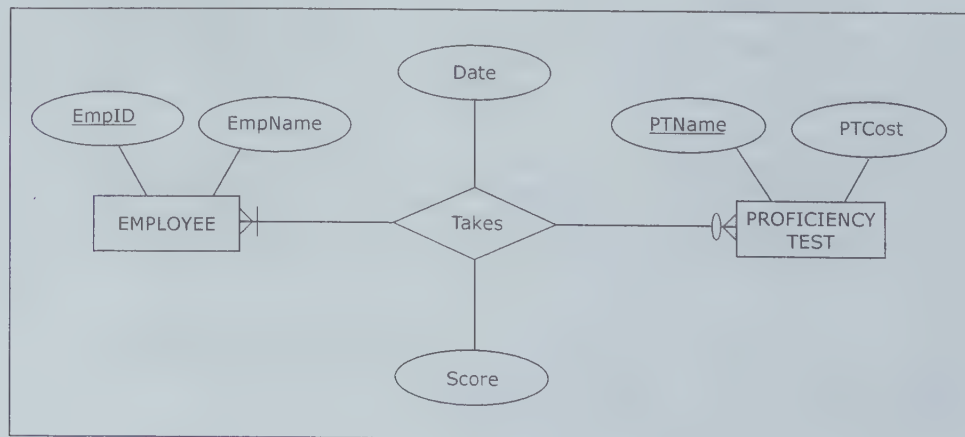


Figure 2.43 An ER diagram for an M:N relationship depicting employees completing proficiency tests

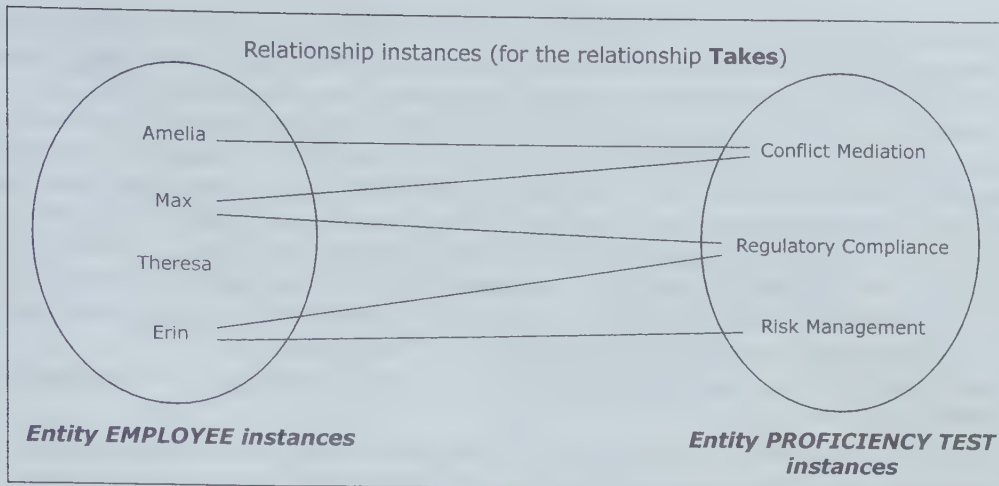


Figure 2.44 Instances of the M:N relationship shown in Figure 2.43

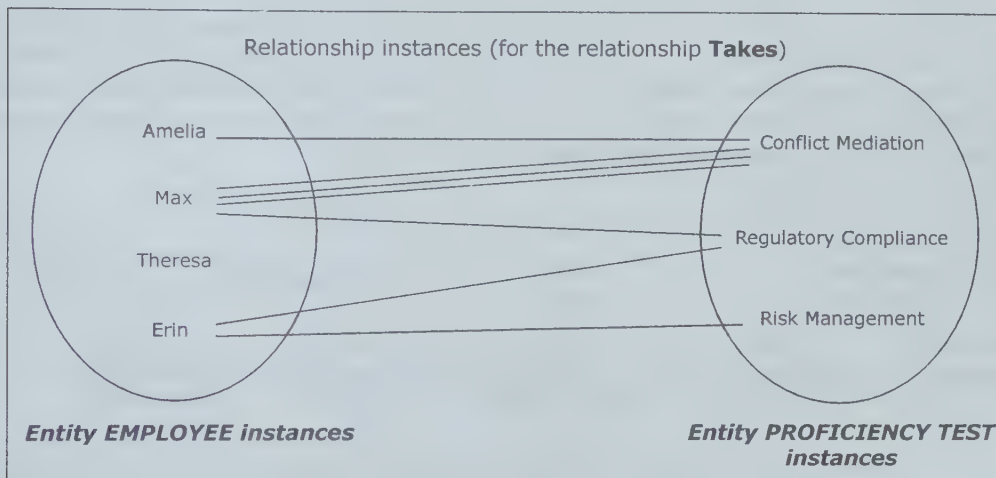


Figure 2.45 Instances of the M:N relationship Takes with an additional requirement

We cannot depict these expanded requirements in an ER diagram as an M:N relationship. Instead, we will use a weak entity as shown in Figure 2.46.

The reason we used a weak entity is to depict the attribute Date as a partial key. Two or more occurrences of the same employee taking the same proficiency test can now be

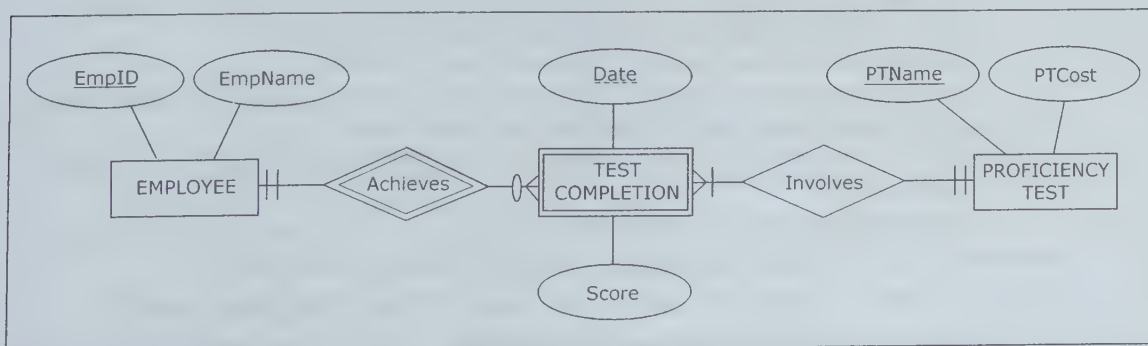


Figure 2.46 An ER diagram for an M:N relationship represented as a weak entity with one owner

distinguished, even if the same score is earned. For example, suppose an employee takes the same proficiency test two times and earns a score of 40 each time, then retakes that same proficiency test for a third time and receives the minimum passing score of 75.

When an M:N relationship is represented as a weak entity, the cardinality constraints of the weak entity are always mandatory-one, and the cardinality constraints of the M:N relationship are transferred to the corresponding non-weak entity. Consider Figures 2.43 and 2.46. The optional-many relationship of EMPLOYEE Takes PROFICIENCY TEST in Figure 2.43 corresponds to the optional-many relationship of EMPLOYEE Achieves TEST COMPLETION in Figure 2.46. Similarly, the mandatory-many relationship of EMPLOYEE Takes PROFICIENCY TEST in Figure 2.43 corresponds to the mandatory-many relationship of TEST COMPLETION Involves PROFICIENCY TEST in Figure 2.46.

We will now make a small change to the newly added requirement. The requirement will now read as follows:

- *An employee can take the same proficiency test more than once (e.g., if an employee receives a score below a minimum score, he or she has to take the same proficiency test again, until the minimum score is achieved). An employee can take multiple proficiency tests on the same date (but not the same proficiency test on the same date).*

To depict this version of the expanded requirements, we will use a weak entity with two owners as shown in Figure 2.47. We need two owners because employees can take multiple tests on the same date. Partial key Date is now unique only if combined with both EmpID and PTName. Note that in Version I, partial key Date was unique when combined with EmpID, because in the first version of added requirements no student was allowed to take multiple proficiency tests on the same date.

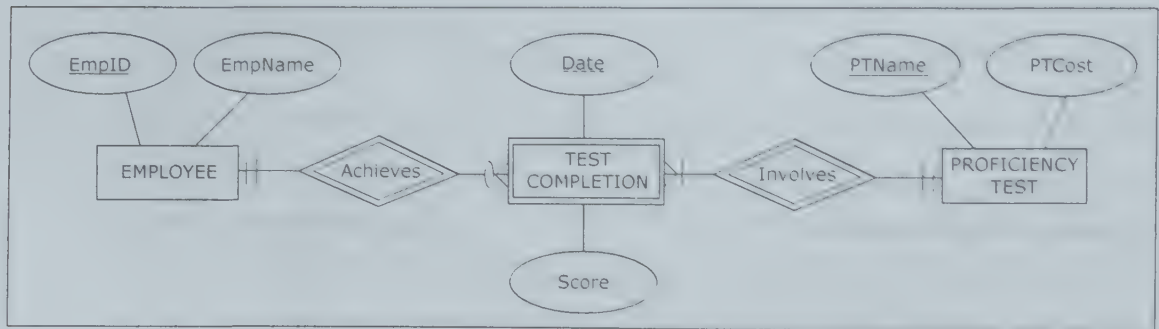


Figure 2.47 An ER diagram for an M:N relationship represented as a weak entity with two owners

Figure 2.48 shows another example of an M:N relationship with multiple instances between the same entity instances, represented as a weak entity. The requirements for this example are as follows.

- *A rental car company wants to keep track of its cars and the customers who rent the cars.*
- *For each customer, we will keep track of his or her unique ID and name.*
- *For each car, we will keep track of its unique VIN and its make.*
- *A customer can rent many cars, but has to rent at least one. A car may be rented by many customers, but we may have cars that have never been rented.*
- *Every time a customer rents a car, we will keep track of the date of rental, length of rental, and price per day.*
- *A customer can rent only one car at a time.*

Note that if we add a small, but critical, addition to these requirements:

- *Each rental that involves one customer renting one car has its own unique rental ID;*

the ER diagram would look as shown in Figure 2.49.

Since each rental now has a unique attribute, we would no longer depict the rental as a weak entity, but as a regular entity.

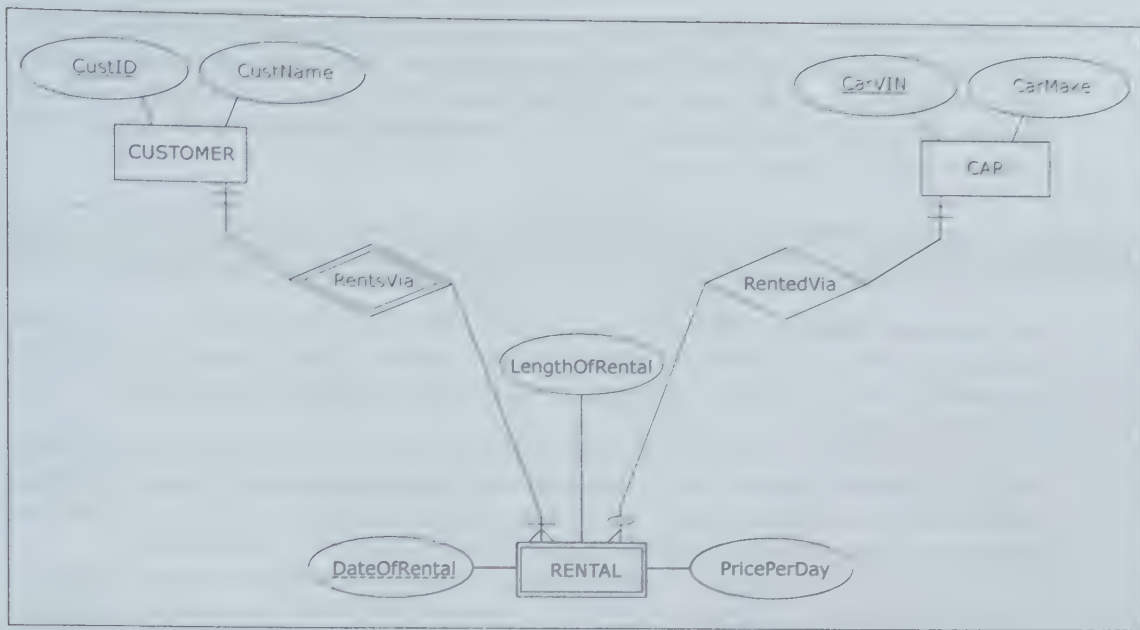


Figure 2.48 Another M:N relationship represented as a weak entity

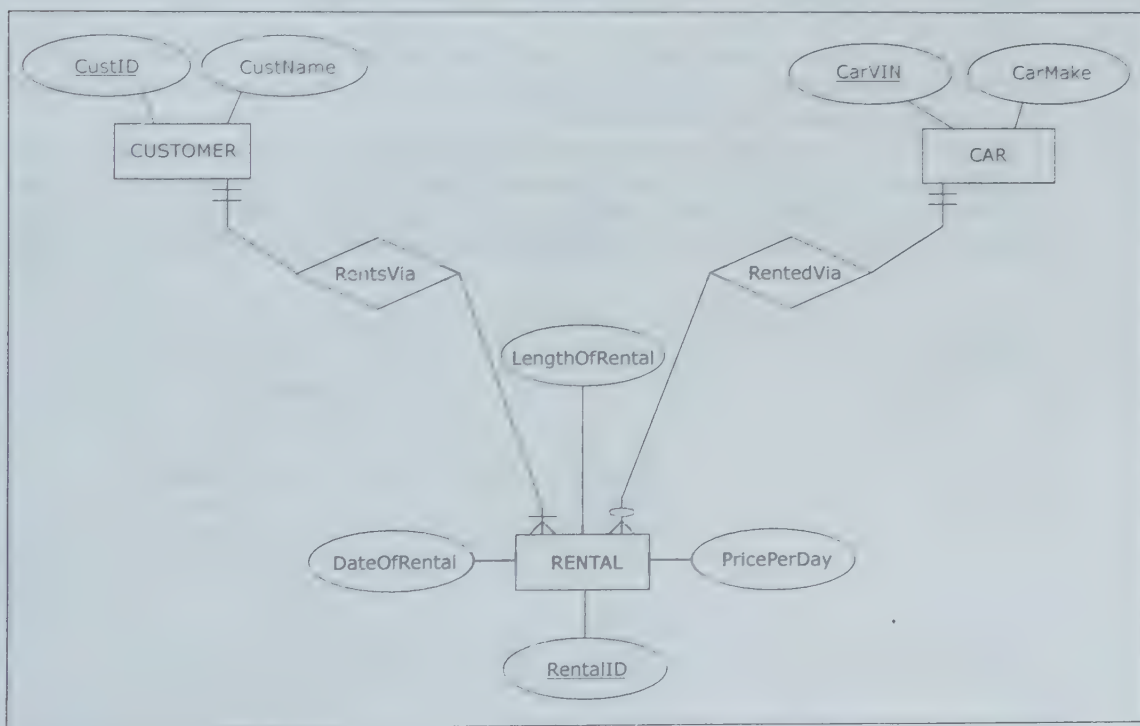


Figure 2.49 A regular entity, instead of an M:N relationship represented as a weak entity

Adding a unique identifier attribute to the requirements for an M:N relationship (especially M:N relationships with attributes) transforms the M:N relationship into a regular entity, as illustrated by the rental car company example in Figure 2.48. Adding the unique identifier in such fashion is a commonly used technique for streamlining ER diagrams and the subsequent resulting databases.

A Note About Associative Entities

An **associative entity** is a construct used as an alternative way of depicting M:N relationships. An associative entity is represented by a rectangle with a diamond inside of it. Associative entities do not have unique or partially unique attributes, and often do not have any attributes at all.

Figure 2.50 shows an M:N relationship and its representation as an associative entity. The top and bottom diagrams in Figure 2.50 are equivalent to each other and are both based on the exact same requirements. Since relationship *AssignedTo* does not have an attribute, the equivalent associative entity *ASSIGNMENT* also does not have an attribute.

Note that entity *EMPLOYEE* has optional participation in the relationship *AssignedTo* in the top diagram in Figure 2.50. Equivalently, entity *EMPLOYEE* has an optional participation in the *Includes* relationship with the *ASSIGNMENT* associative entity in the bottom diagram in Figure 2.50. On the other hand, entity *PROJECT* has a mandatory participation in the relationship *AssignedTo* in the top diagram in Figure 2.50. Equivalently, entity *PROJECT* has a mandatory participation in the *IsFor* relationship with the *ASSIGNMENT* associative entity in the bottom diagram in Figure 2.50. Also note that the cardinality constraints for the associative entity itself in both relationships are mandatory-one (this is always the case with associative entities).

Figure 2.51 shows a unary M:N relationship and its representation as an associative entity. The top and bottom diagrams in Figure 2.51 are equivalent to each other and are both based on the exact same requirements.

Figure 2.52 shows an M:N relationship with an attribute and its representation as an associative entity. The top and bottom diagrams in Figure 2.52 are equivalent to each other and are both based on the exact same requirements. Since relationship *Includes* has an attribute *Quantity* in the top diagram, the equivalent associative entity *LINEITEM* also has

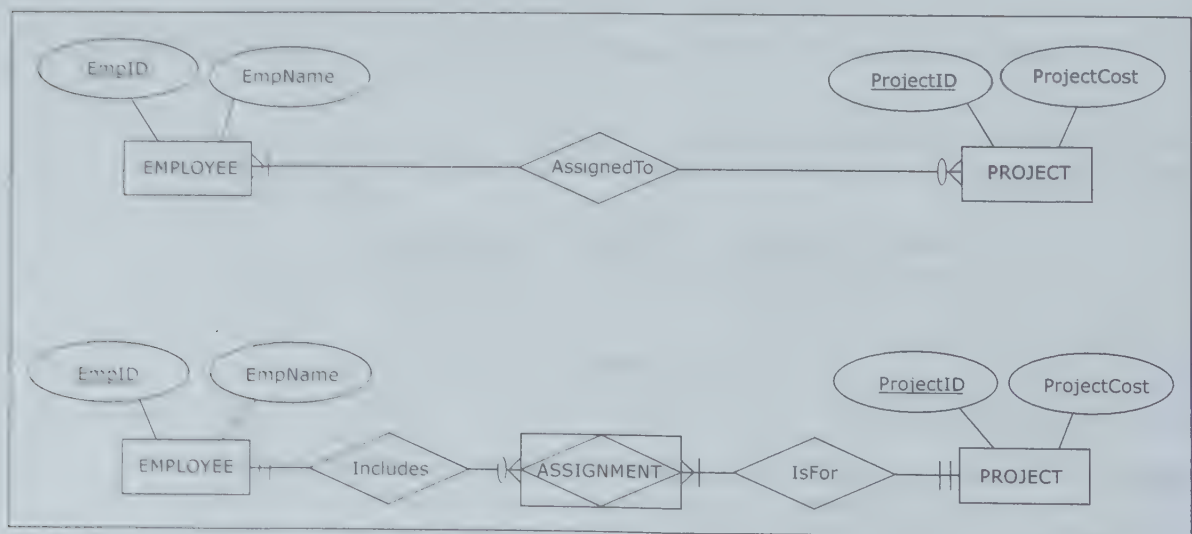


Figure 2.50 An identical relationship represented as an M:N relationship and as an associative entity

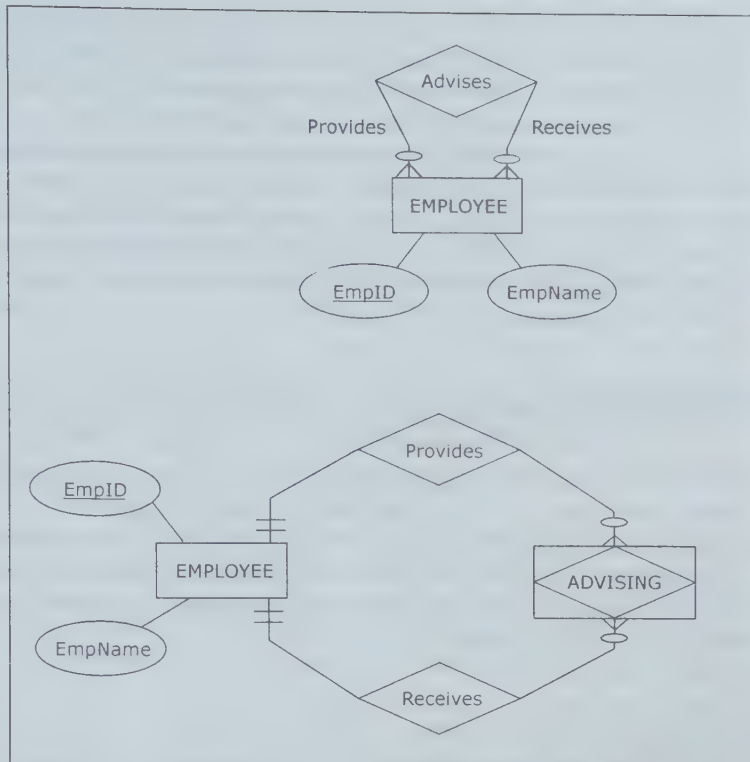


Figure 2.51 An identical relationship represented as a unary M:N relationship and as an associative entity

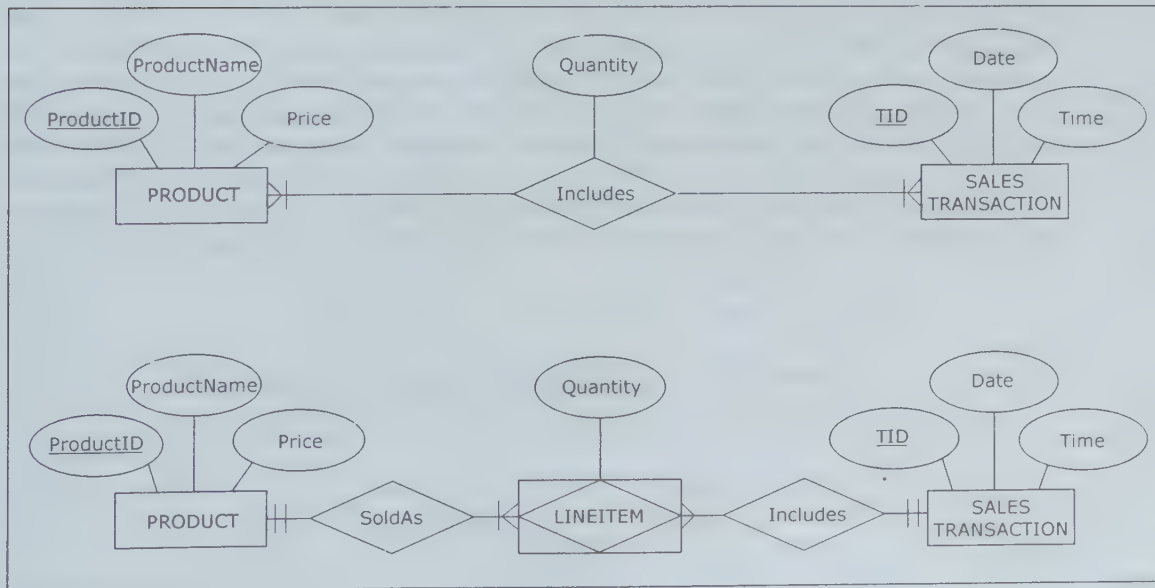


Figure 2.52 An identical relationship represented as an M:N relationship with an attribute and as an associative entity with an attribute.

the same attribute Quantity in the bottom diagram. Each sales transaction can include many products in different quantities, as shown by the top diagram. Therefore, in the bottom diagram, each sales transaction can have many line items, where each line item represents the sale of one product and its quantity within one sales transaction.

Associative entities are not necessary constructs for depicting binary or unary relationships. As we illustrated in the above examples, for binary or unary relationships, associative entities are simply another way of depicting a relationship that can just as easily be depicted without using an associative entity. However, as we show in the next note, for relationships with a degree higher than 2, such as ternary relationships, associative entities provide a way to eliminate potential ambiguities in the ER diagrams.

A Note About Ternary (and Higher Degree) Relationships

A relationship of degree 3, involving three entities, is also known as a **ternary relationship**. To illustrate a ternary relationship, we will use the following example:

Company MG (Manufacturing Guru) wants to keep track of its suppliers, components, and products. In particular, Company MG wants to keep track of which supplier provides which component for which product. During the requirements collections process, Company MG provided the following specifics:

- We have multiple products.
- We have multiple suppliers.
- We have multiple components.
- We will keep track of which suppliers provide which components for which product.
- Every product contains one or more components, each of which is provided by one or more suppliers.
- Every supplier provides between zero and many components for use in between zero and many products.
- Every component is provided for one or many products by one or many suppliers.

Simply creating three binary relationships between the three entities *will not* fully depict the above requirements. Figure 2.53 shows three relationships depicting which product is associated with which supplier, which supplier is associated with which component, and which component is associated with which product (for brevity, all figures in this section are shown without entity attributes). However, the database based on the diagram in Figure 2.53 *would not* keep track of which suppliers provide which components for which product.

For example, in the database resulting from the diagram, we could depict that suppliers A1 and A2 are the providers of component B, and that component B is one of the

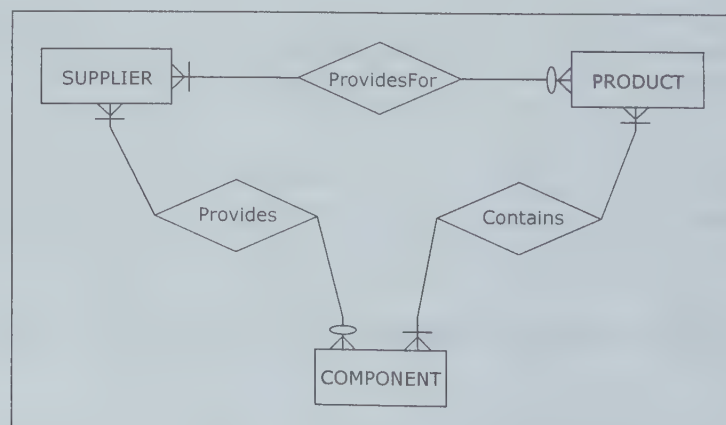


Figure 2.53 Three binary relationships that are insufficient for depicting Company MG requirements.

components for product C. However, let us assume that supplier A1 provides component B for product C, and that the supplier A2 does not provide component B for product C (i.e., supplier A2 provides component B, but not for product C). The database resulting from diagram 2.52 could not depict such a scenario.

In order to accommodate such perfectly plausible scenarios, all three entities must be connected with one relationship. In other words, a ternary relationship has to be created.

One problematic issue with ternary relationships is that it is not possible to unambiguously show cardinality constraints in a ternary relationship in all notations. Observe Figure 2.54, showing the ternary relationship Provides that involves entities SUPPLIER, PRODUCT, and COMPONENT. The reason there are no cardinality constraints in this figure is that it is not possible to assign cardinality constraints unambiguously using our chosen notation.

It is not clear in the ER diagram in Figure 2.54 where we would put a symbol indicating that we may record some suppliers who do not provide any components for any products. If we put an optional symbol on the component side of the relationship, it is simply not clear if we are saying that we want to keep track of suppliers that do not provide any components for products, or if we are saying that we want to keep track of products for which no components are provided by any suppliers.

In other words, we simply cannot depict cardinalities unambiguously in a ternary relationship. However, if we use an associative entity instead, as shown in Figure 2.55, then we can depict the ternary relationship with all of its necessary cardinality constraints unambiguously.

There is another approach to depicting all instances of suppliers providing components for products. If we created a unique attribute SPC (Supplier providing a Component for a Product) code, and associated a unique value of it with each separate instance of a supplier providing a component for a product, we would avoid the need for a ternary relationship,

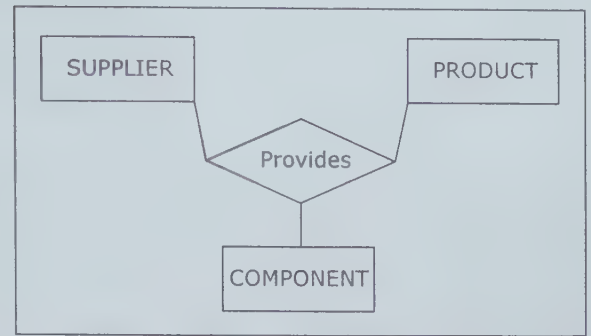


Figure 2.54 A ternary relationship

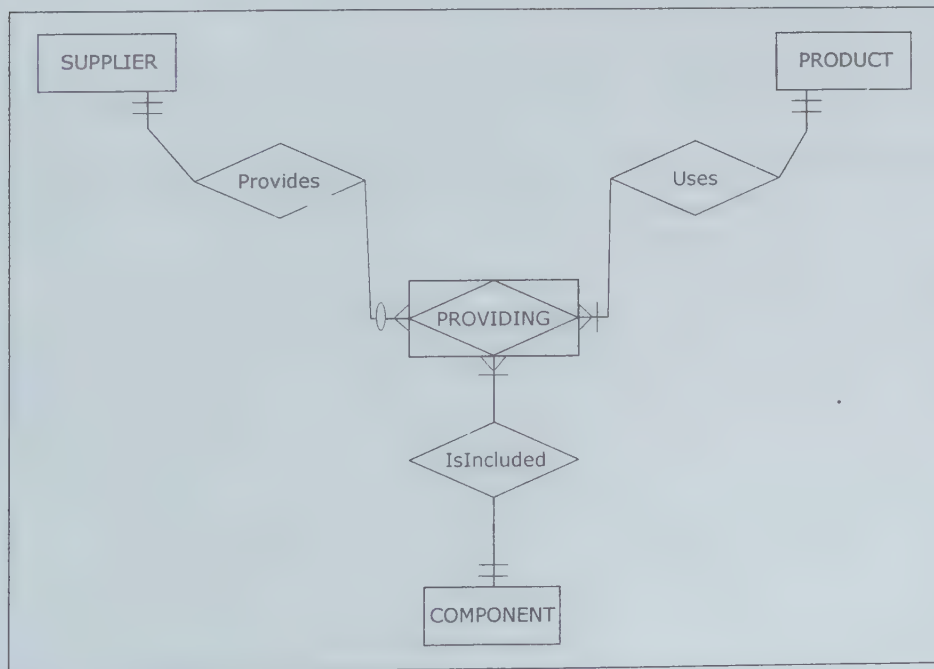


Figure 2.55 A ternary relationship via associative entity

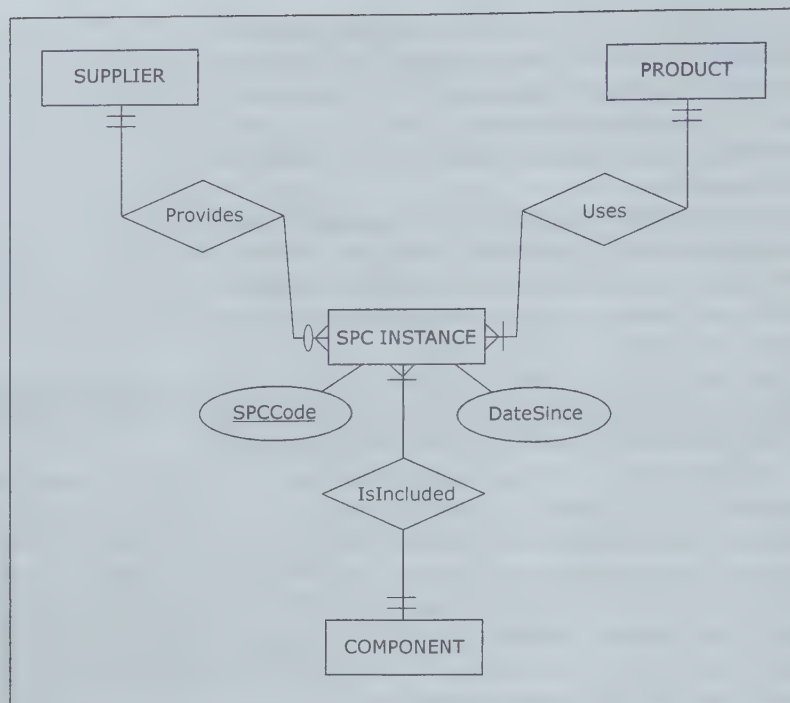


Figure 2.56 A regular entity replacing a ternary relationship

as shown in Figure 2.56. In this figure, ternary relationship is replaced by a regular entity SPC INSTANCE. In addition, we are now also recording when a particular entity started providing a particular component for a particular product.

Another thing to note about ternary relationships is that, in addition to being rare, they are applicable as many-to-many-to-many relationships. To illustrate, consider Figure 2.57, which instead of a many-to-many-to-many relationship, shows a many-to-many-to-one ternary relationship.

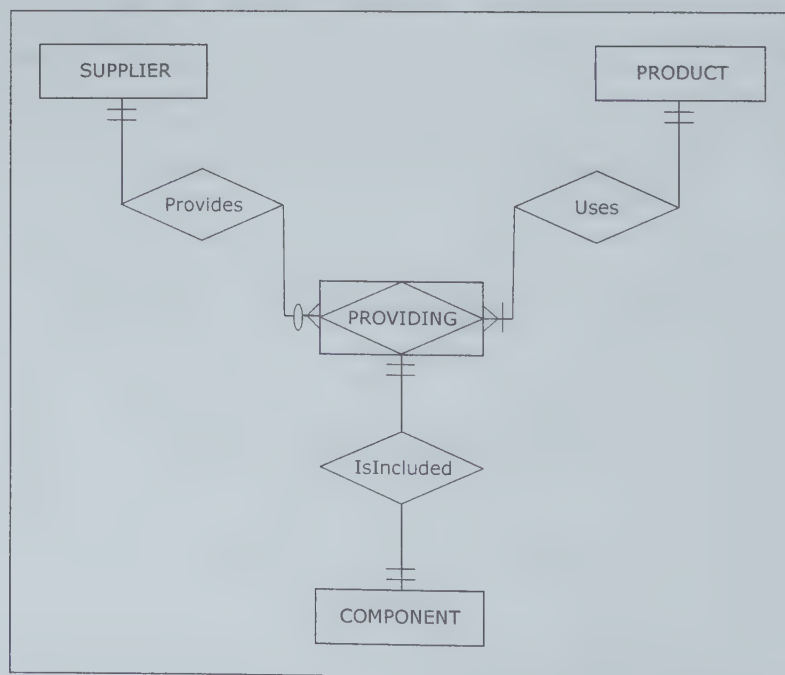


Figure 2.57 A many-to-many-to-one ternary relationship

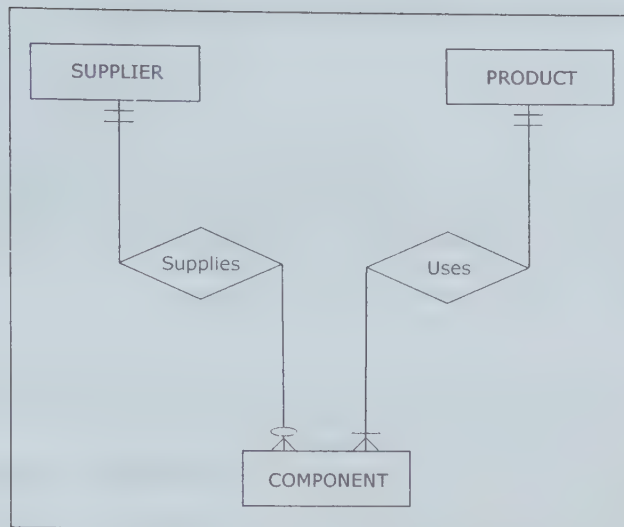


Figure 2.58 A many-to-many to-one ternary relationship eliminated

The requirements for this diagram state that *each component is exclusively supplied by one supplier for one product*. Therefore, the diagram could have been simplified as shown in Figure 2.58, eliminating the need for a ternary relationship.

In practice, ternary relationships are relatively rare, and relationships of degree higher than 3 are rarer still. In most cases when a designer is tempted to create relationships of degrees higher than 2, he or she should explore the possibility of creating additional entities instead.

A Note About Reducing the Number of Attributes by Treating Some Attributes as Attribute Values

In some cases, it may be appropriate to consider whether an originally designed attribute can be treated as an attribute value instead. Consider the following example:

For each employee, we will keep track of his or her unique ID and name as well as their bonuses for Quarter 1, Quarter 2, Quarter 3, and Quarter 4.

One way to represent this requirement is shown in Figure 2.59. In this figure, entity EMPLOYEE has six attributes.

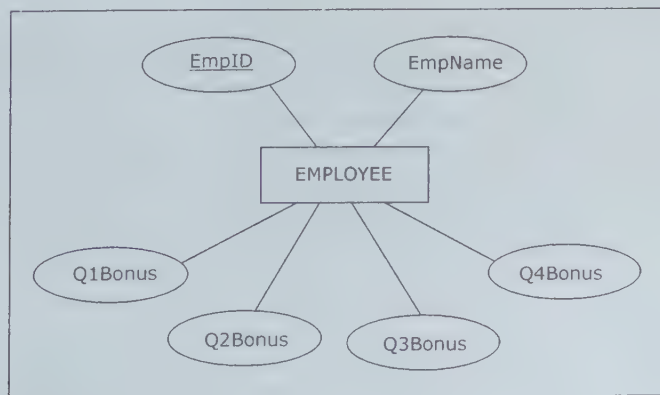


Figure 2.59 An entity with several attributes

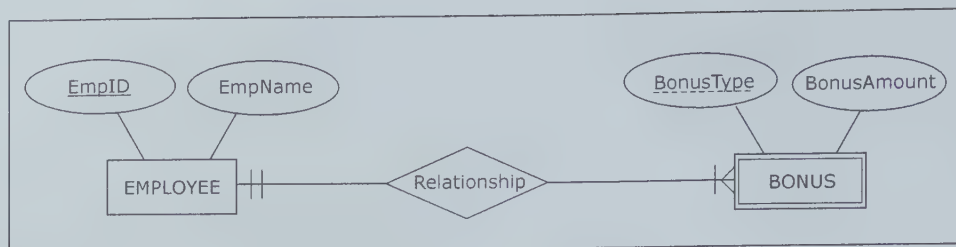


Figure 2.60 Alternative design of the entity in Figure 2.59

An alternative way of representing these requirements is shown in Figure 2.60. Multiple different bonus attributes are replaced with two attributes (Bonus Type and Bonus Amount) in a newly created weak entity BONUS.

Note that the exact same information can be accommodated by databases based on ER Diagrams in Figures 2.59 and 2.60.

Consider an example of an employee Joe whose id is 111 and whose quarter 1 bonus is \$100, quarter 2 bonus is \$200, quarter 3 bonus is \$300, and quarter 4 bonus is \$400. To depict this scenario, the entity instance of entity EMPLOYEE in Figure 2.59 would have the following values:

EmpID: 111, EmpName: Joe, Q1Bonus: 100, Q2Bonus: 200, Q3Bonus: 300 and Q4Bonus: 400.

The same scenario depicted in Figure 2.60 would involve one entity instance of the entity EMPLOYEE with values:

EmpID: 111, EmpName: Joe,

as well as four entity instances of the weak entity BONUS with the following values:

BonusType: Q1, BonusAmount: 100;
 BonusType: Q2, BonusAmount: 200;
 BonusType: Q3, BonusAmount: 300;
 BonusType: Q4, BonusAmount: 400.

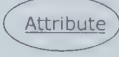
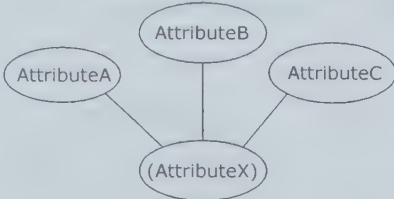
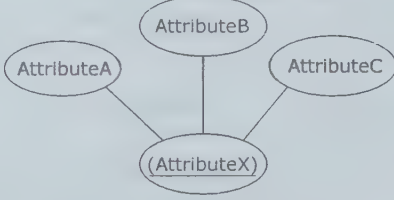
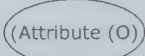
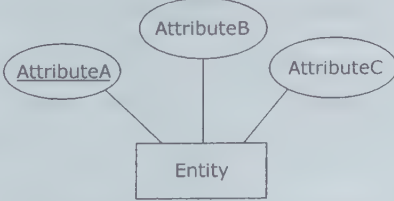
In the regular approach, illustrated by Figure 2.59, there are more attributes and all attribute metadata is captured by attribute names. In the alternative approach, illustrated by Figure 2.60, there are fewer attributes but some of the original metadata (such as Q1Bonus, Q2Bonus, Q3Bonus, and Q4Bonus) is now converted to data of a new column (BonusType values Q1, Q2, Q3, and Q4).

In most cases, the regular approach is both intuitive and sufficient, but in some circumstances the alternative approach is a valid alternative. For example, if every employee was given a bonus for each quarter, then the regular approach works fine. On the other hand, if a company wants to keep track of quarterly bonuses for a number of years, and those bonuses were optional and given to a small percentage of employees, then the alternative approach may work better, because it would eliminate the need for a growing number of attributes (that would, for the majority of entity instances, contain empty values).

Summary

Table 2.1 provides a summary of the basic ER modeling concepts introduced in this chapter.

Table 2.1 Summary of the Basic ER Modeling Concepts

Concept	Graphical Presentation
Regular Attribute	
Unique Attribute	
Composite Attribute	
Composite Unique Attribute	
Multi-Valued Attribute	
Derived Attribute	
Optional Attribute	
Entity with Attributes Notes: Each entity must have at least one unique attribute. Within one entity each attribute name is different. Within one ER Diagram each entity name is different	

Concept	Graphical Presentation
Relationship	
Cardinality Constraints	Participation (Minimum Cardinality)—Optional or Mandatory Maximum Cardinality—One or Many
Four Possible Cardinality Constraints	Mandatory Many Optional Many Mandatory One Optional One
Three Types of Relationships (Maximum Cardinality-wise)	One to One One to Many (Many to One) Many to Many
Relationship with an Attribute Note: Applicable for many to many relationships	
Relationship with specific Minimum and Maximum Cardinalities	

Concept	Graphical Presentation
Unary Relationship Degree of a Relationship: 1 (One entity involved)	
Binary Relationship Degree of a Relationship: 2 (Two entities involved)	
Ternary Relationship Degree of a Relationship: 3 (Three entities involved) Notes: Applicable as many-to-many relationships Ternary relationships are rare (relationships of a degree higher than 3 are even rarer).	
Weak Entity Notes: Always associated with its owner via an identifying relationship (which is either 1:M or 1:1) If identifying relationship is 1:M, then weak entity must have a partially unique attribute.	
Associative Entity Notes: Alternative method for depicting M:N relationships; particularly useful for depicting ternary relationships.	

Key Terms

Associative entity
 Attribute (of an entity)
 Binary relationship
 Candidate key
 Cardinality constraints
 Composite attribute
 Composite unique attribute
 Database requirements
 Degree of a relationship
 Derived attribute
 Enhanced ER (EER)
 Entity

Entity instances (entity members)
 Entity-relationship (ER) modeling
 ER diagram (ERD)
 Exact minimum and/or maximum cardinality
 Identifying relationship
 Many-to-many relationship (M:N)
 Maximum cardinality
 Minimum cardinality (participation)
 Multivalued attribute
 One-to-many relationship (1:M)
 One-to-one relationship (1:1)

Optional attribute
 Owner entity
 Partial key
 Relationship
 Relationship attributes
 Relationship instances
 Relationship role
 Ternary relationship
 Unary relationship (recursive relationship)
 Unique attribute
 Weak entity

Review Questions

- | | | | |
|-------------|---|--------------|---|
| Q2.1 | What is the purpose of ER modeling? | Q2.9 | What is a multivalued attribute? |
| Q2.2 | What are the basic ER modeling constructs? | Q2.10 | What is a derived attribute? |
| Q2.3 | What is a unique attribute? | Q2.11 | What is an optional attribute? |
| Q2.4 | What is depicted by cardinality constraints? | Q2.12 | How are exact minimum and maximum cardinalities depicted in a relationship? |
| Q2.5 | What are the four possible cardinality constraints? | Q2.13 | What is a binary relationship? |
| Q2.6 | What are the three types of relationships (maximum cardinality-wise)? | Q2.14 | What is a unary relationship? |
| Q2.7 | What is a composite attribute? | Q2.15 | What is a weak entity? |
| Q2.8 | What are candidate keys? | Q2.16 | What is an associative entity? |
| | | Q2.17 | What is a ternary relationship? |

Exercises

E2.1 Create an entity based on the following scenarios.

E2.1a Create an entity based on the following requirement:

- For each doctor, we will keep track of a *DoctorID* (unique), *DoctorName*, and *DoctorYearOfMDGraduation* (year of graduating from medical school).

E2.1b Create an entity based on the following requirement:

- For each doctor, we will keep track of a *DoctorID* (unique), *DoctorName* (composed of first and last name), and *DoctorYearOfMDGraduation* (year of graduating from medical school).

E2.1c Create an entity based on the following requirement:

- For each doctor, we will keep track of a *DoctorID* (unique), *DoctorLicense* (unique, composed of *LicenseNo* and *State*), *DoctorName*, and *DoctorYearOfMDGraduation* (year of graduating from medical school).

E2.1d Create an entity based on the following requirement:

- For each doctor, we will keep track of a *DoctorID* (unique), *DoctorName*, *DoctorYearOfMDGraduation* (year of graduating from medical school), and one or more specializations.

E2.1e Create an entity based on the following requirement:

- For each doctor, we will keep track of a *DoctorID* (unique), *DoctorName*, *DoctorYearOfMDGraduation* (year of graduating from medical school), and *DoctorYearsPracticing* (how many years the doctor has been practicing medicine).

E2.1f Create an entity based on the following requirement:

- For each doctor, we will keep track of a *DoctorID* (unique), *DoctorName*, *DoctorYearOfMDGraduation* (year of graduating from medical school), and *DoctorYearOfPhDGraduation* (year of acquiring a PhD degree; not every doctor has a PhD degree).

E2.2 Create the ER diagrams for a scenario based on these requirements:

- A health care organization keeps track of its doctors and outpatient locations.
- For each doctor it keeps track of the *DoctorID* (unique), *DoctorName*, and *DoctorYearOfMDGraduation* (year of graduating from medical school).
- For each outpatient location it keeps track of the *OLID* (unique) and *OLName*,

and the following additional requirement:

E2.2a Additional requirement

- Each doctor works at exactly one outpatient location, and each outpatient location can have between none (rehab and minor health issues only) and many doctors working at it.

E2.2b Additional requirement

- Each doctor works at either one outpatient location or at none (strictly working in the main hospital), and each outpatient location must have at least one but can have many doctors working at it.

E2.2c Additional requirement

- Each doctor works at exactly one outpatient location, and each outpatient location must have at least one but can have many doctors working at it.

E2.2d Additional requirement

- Each doctor works at either one outpatient location or at none (strictly working in the main hospital), and each outpatient location can have between none (rehab and minor health issues only) and many doctors working at it.

E2.2e Additional requirement

- Each doctor works at between none (strictly working in the main hospital), and many outpatient locations, and each outpatient location must have at least one but can have many doctors working at it.

E2.2f Additional requirement

- Each doctor works at between one and many outpatient locations, and each outpatient location must have one but can have many doctors working at it.

E2.2g Additional requirement

- Each doctor works at between none (strictly working in the main hospital) and many outpatient locations, and each outpatient location can have between none (rehab and minor health issues only) and many doctors working at it.

E2.2h Additional requirement

- Each doctor works at either one outpatient location or at none (strictly working in the main hospital), and each outpatient location has exactly one doctor working at it.

E2.2i Additional requirement

- Each doctor works at exactly one outpatient location, and each outpatient location has exactly one doctor working at it.

E2.2j Additional requirement

- Each doctor works at either one outpatient location or at none (strictly working in the main hospital), and each outpatient location can have between none (rehab and minor health issues only) and one doctor working at it.

E2.2k Additional requirements

- Each doctor works at between one and many outpatient locations, and each outpatient location must have at least one but can have many doctors working at it.
- For each occurrence of a doctor working at an outpatient location, we keep track of the NoOfHoursPerWeek (number of hours per week that a doctor works at an outpatient location).

E2.2l Additional requirement

- Each doctor works at between two and four outpatient locations, and each outpatient location has between three and ten doctors working at it.

E2.2m Additional requirements

- Each doctor works at between one and many outpatient locations, and each outpatient location must have at least one but can have many doctors working at it.
- Each doctor manages between zero and one outpatient locations, and each outpatient location has exactly one doctor managing it.

E2.3 Create the ER diagram based on the following requirements:

- A health care organization keeps track of its doctors.
- For each doctor it keeps track of the DoctorID (unique), DoctorName, and DoctorYearOfMDGraduation (year of graduating from medical school).
- A doctor can be a resident for either one or none of the faculty doctors. A doctor can have many resident doctors, but does not have to have any.

E2.4 Create the ER diagram based on the following requirements:

- A health care organization keeps track of outpatient locations and examination rooms within the outpatient locations.
- For each outpatient location it keeps track of the OLID (unique) and OLName.
- For each examination rooms it keeps track of the ERoomNumber (unique within its outpatient center, but examination rooms in different outpatient locations can have the same ERoomNumber), and ERoomSize.
- Each outpatient location must have at least one but can have many examination rooms. Each examination room is located in one outpatient location.

E2.5 Create your own example of an entity**E2.5a** Entity with several attributes.**E2.5b** Entity with several attributes, where one of the attributes is a composite attribute.**E2.5c** Entity with several attributes, where one of the attributes is a composite unique attribute.**E2.5d** Entity with several attributes and more than one unique attribute.**E2.5e** Entity with several attributes, where one of the attributes is a multivalued attribute.**E2.5f** Entity with several attributes, where one of the attributes is a derived attribute.**E2.5g** Entity with several attributes, where one of the attributes is an optional attribute.**E2.6** Create your own requirements and the ER diagram for a scenario with two entities (both with several attributes) involved in:**E2.6a** 1:M relationship, where participation on the 1 side is mandatory and participation on the M side is optional.

- E2.6b** 1:M relationship, where participation on the 1 side is optional and participation on the M side is mandatory.
- E2.6c** 1:M relationship, where participation on both sides is mandatory.
- E2.6d** 1:M relationship, where participation on both sides is optional.
- E2.6e** M:N relationship, where participation on one side is mandatory and participation on the other side is optional.
- E2.6f** M:N relationship, where participation on both sides is mandatory.
- E2.6g** M:N relationship, where participation on both sides is optional.
- E2.6h** 1:1 relationship, where participation on one side is mandatory and participation on the other side is optional.
- E2.6i** 1:1 relationship, where participation on both sides is mandatory.
- E2.6j** 1:1 relationship, where participation on both sides is optional.
- E2.6k** M:N relationship, where the relationship has its own attribute.
- E2.6l** M:N relationship, with exact minimum and maximum cardinalities on both sides.
- E2.6m** Two separate relationships.
- E2.7** Create requirements and the ER diagram with several attributes for the scenario with an entity involved in a unary relationship.
- E2.8** Create requirements and the ER diagram for the scenario with two entities (both with several attributes), one of which is a weak entity and the other its owner entity, involved in an identifying relationship.

Mini Cases

MC1 Advance Science Incorporated

Advance Science Incorporated is a science and technology company. Create the ER diagram for the Advance Science Inc. Database, based on the following requirements.

- The Advance Science Incorporated Database will keep track of employees, facilities, and devices.
- For each employee, Advance Science Incorporated will keep track of a unique employee identifier as well as his or her name.
- For each facility, Advance Science Incorporated will keep track of a unique facility identifier as well as the facility type.
- For each device, Advance Science Incorporated will keep track of a unique device identifier as well as the device type.
- Each employee can access one or more facilities. Each facility can be accessed by one or more employees.
- Each device is issued to exactly one employee. Each employee can have between zero and many devices issued to him or her.

MC2 Investco Scout

Investco Scout is an investment research company. Create the ER diagram for the Investco Scout Funds Database, based on the following requirements.

- The Investco Scout Funds Database will keep track of investment companies, the mutual funds they manage, and securities contained in the mutual funds.
- For each investment company, Investco Scout will keep track of a unique investment company identifier and a unique investment company name as well

as the names of multiple locations of the investment company.

- For each mutual fund, Investco Scout will keep track of a unique mutual fund identifier as well as the mutual fund name and inception date.
- For each security, Investco Scout will keep track of a unique security identifier as well as the security name and type.
- Investment companies can manage multiple mutual funds. Investco Scout does not keep track of investment companies that do not manage any mutual funds. A mutual fund is managed by one investment company.
- A mutual fund contains one or many securities. A security can be included in many mutual funds. Investco Scout keeps track of securities that are not included in any mutual funds.
- For each instance of a security included in a mutual fund, Investco Scout keeps track of the amount included.
- For each investment company, Investco Scout keeps track of how many mutual funds they manage.

MC3 Funky Bizz

Funky Bizz is a rental business that rents musical instruments to bands. Create the ER diagram for the Funky Bizz Operations Database, based on the following requirements.

- The Funky Bizz Operations Database will keep track of instruments, bands, repair technicians, and shows.

- For each instrument, Funky Bizz will keep track of a unique instrument serial number as well as the instrument model and brand, the year when the instrument was made, and the age (measured in years) of the instrument.
- The customers of Funky Bizz are bands. For each band, Funky Bizz will keep track of the unique band name and unique band identifier as well as the band's address, contact person's name, and multiple phone numbers.
- Repair technicians maintain the instruments. For each technician, Funky Bizz will keep track of a unique SSN as well as a name, address, and multiple phone numbers.
- Funky Bizz will record information about shows that its customer bands perform in. For each show, it will keep track of a unique show identifier composed of the show venue name and date. For each show, it will also keep track of show type and show name (a show may or may not have a name).
- A band does not have to rent any instruments, but may rent up to 30. Each instrument may be rented by one band or by no bands at all.
- A repair technician maintains one or many instruments. Each instrument is maintained by one technician.
- A band may perform in many shows, but does not have to perform in any. Each show must have at least one band performing, but may have many bands performing. For each instance of a band performing in a show, we keep track of the duration (in minutes) of that band's performance in that show.
- For each band, Funky Bizz keeps track of the number of shows that the band performs in.
- For each show, Funky Bizz keeps track of the number of bands that participated in the show.

MC4 Snooty Fashions

Snooty Fashions is an exclusive custom fashion designer business. Create the ER diagram for the Snooty Fashions Operations Database based on the following requirements.

The Snooty Fashions Operations Database will keep track of the following:

- For each designer: a unique designer identifier and unique SSN as well as the name (composed of first and last name);
- For each customer: a unique customer's identifier as well as his or her name and multiple phone numbers;
- For each tailoring technician: a unique SSN as well as his or her name (composed of first and last name);
- For each outfit: a unique outfit's identifier as well as the outfit's planned date of completion and its unreasonable price;

- For each fashion show: a unique show identifier as well as the date of the show and location;
- Each designer designs many outfits. Each outfit has only one designer.
- Each outfit is sold (in advance) to exactly one customer. Customers can buy one or many outfits (Snooty Fashions will not keep track of customers who have not made any purchases yet).
- Each tailoring technician must work on at least one outfit but can work on many. Each outfit has at least one tailoring technician working on it but can have many.
- Snooty Fashions will keep track of the date when a tailoring technician started working on a particular outfit.
- Each designer can participate in a number of fashion shows, but does not have to participate in any. Each fashion show can feature one or two Snooty Fashions designers. (Snooty Fashions will not keep track of fashion shows that do not feature Snooty Fashions designers.)

MC5 Signum Libri

Signum Libri (SL) is a publishing company. Create the ER diagram for the SL Operations Database based on the following requirements.

SL Operations Database will keep track of the following:

- For each book SL publishes: a book name, genre, date of publication, and number of pages;
- For each writer: a unique writer identifier as well as the writer's name;
- For each agent: a unique agent identifier as well as the agent's name;
- For each editor: a unique editor identifier as well as the editor's name;
- Each SL book is written by one writer, and each writer can write many SL books. SL will not keep track of writers who did not write a book for SL. All books written by the same writer have a different book name. However, two writers can write two different books with the same book name.
- Each writer is represented by one agent. Each agent represents at least one writer, but can represent many.
- Each book has one editor. Each editor edits at least one book, but can edit many books.
- Each editor can mentor one or more other editors, but does not have to mentor any. Each editor can have at most one mentor editor, but does not have to have any.

MC6 Ava's Consulting

Ava's Consulting (AC Inc) is a consulting company. Create the ER diagram for the AC Inc Operations Database based on the following requirements.

AC Inc Operations Database will keep track of the following:

- For each consultant: a unique cid, as well as the consultant's name, one or more skills, start date with the company, and the number of years with company;
- For each project manager: a unique pmid, the project manager's name;
- For each customer: a unique custid, well as the customer's name composed of customer's first and last name, and a phone number;
- For each supervisor: a unique supid, as well as the supervisor's name, number of supervised contracts, and (if supervisor owns AC Inc. stock) number of stocks owned. (Not every supervisor owns AC Inc. stock.)
- For each contract: a contract id and contract start date. Two or more contracts can have the same contract id, but all contracts for the same customer will have different contract ids.
- Each contract is managed by one project manager. A project manager manages one contract or no contracts at all.
- Each contract has at least three consultants assigned to it. A consultant is assigned to one contract.
- A customer has at least one contract but may have many. Each contract is associated with one customer.

- Each contract is supervised by one supervisor. Each supervisor must supervise at least one contract but can supervise many contracts.
- Each supervisor has one or more consultants assigned as administrative helpers. A consultant can be assigned as administrative helper to between zero and many supervisors. We keep track of how many hours per week a particular consultant is working as an administrative helper for a particular supervisor.

MC7 ExoProtect

ExoProtect is an insurance company. Write out all requirements for the ER diagram for the ExoProtect Employees' Computers Database shown in Figure 2.61.

MC8 Jones Dozers

Jones Dozers is a construction equipment company. Write out all requirements for the ER diagram for the Jones Dozers Sales and Rentals Database shown in Figure 2.62.

MC9 City Boomers

City Boomers is a professional soccer team. Write out all requirements for the ER diagram for the City Boomers Revenue Database shown in Figure 2.63.

MC10 Midtown Memorial

Midtown Memorial is a hospital. Write out all requirements for the ER diagram for the Midtown Memorial Patients Drug Dispense Database shown in Figure 2.64.

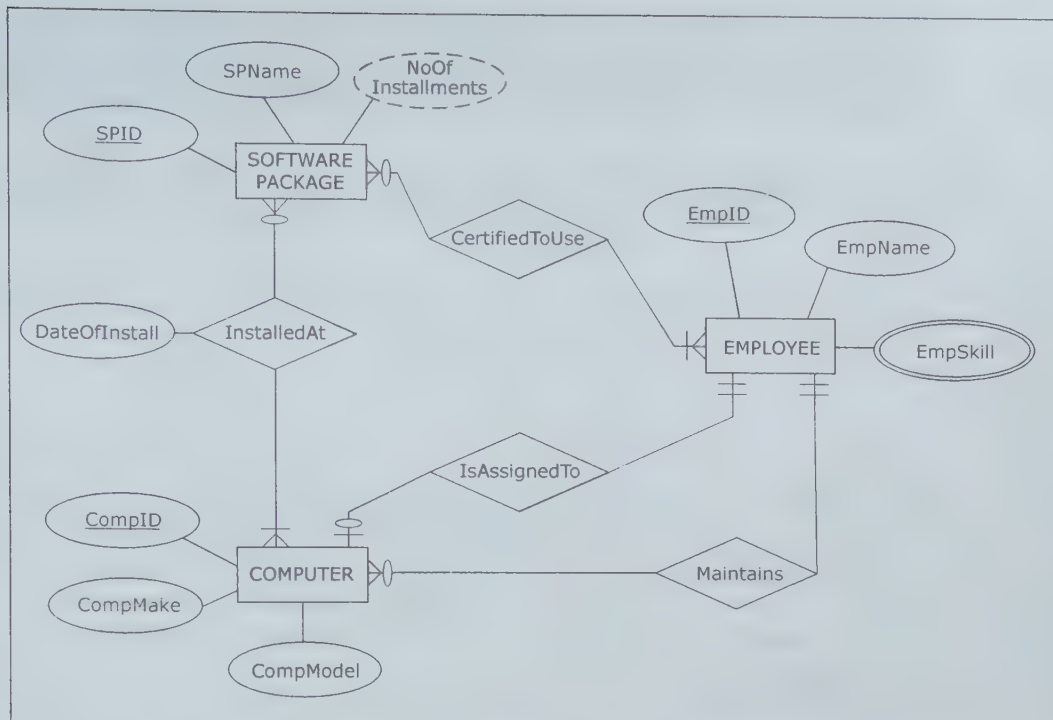


Figure 2.61 An ER diagram for the ExoProtect Employees' Computers Database

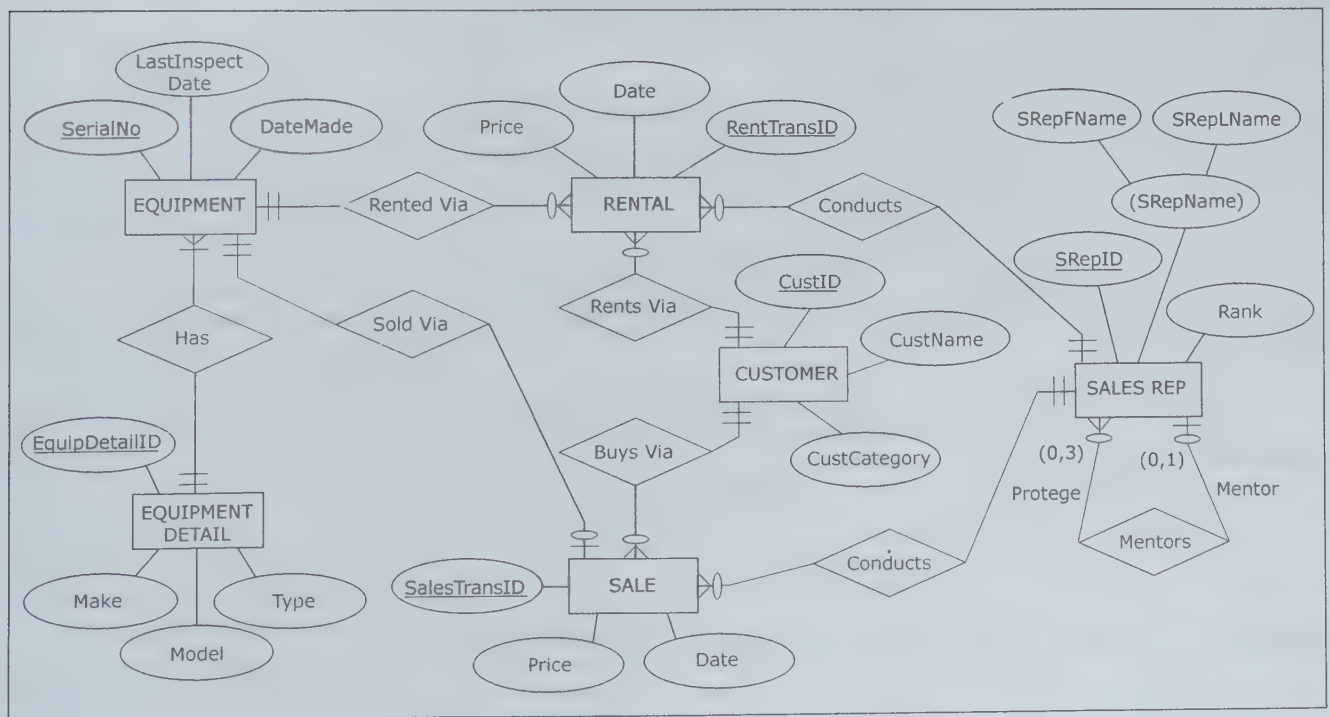


Figure 2.62 ER diagram for the Jones Dozers Sales and Rentals Database

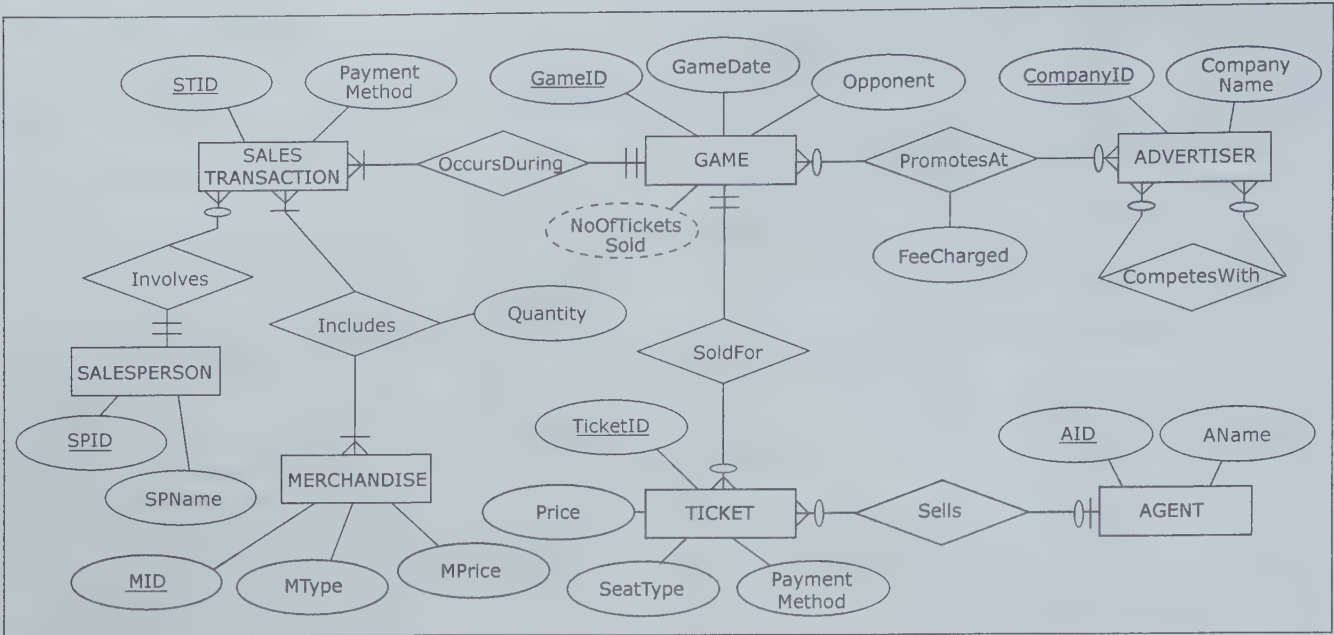


Figure 2.63 ER diagram for the City Boomers Revenue Database

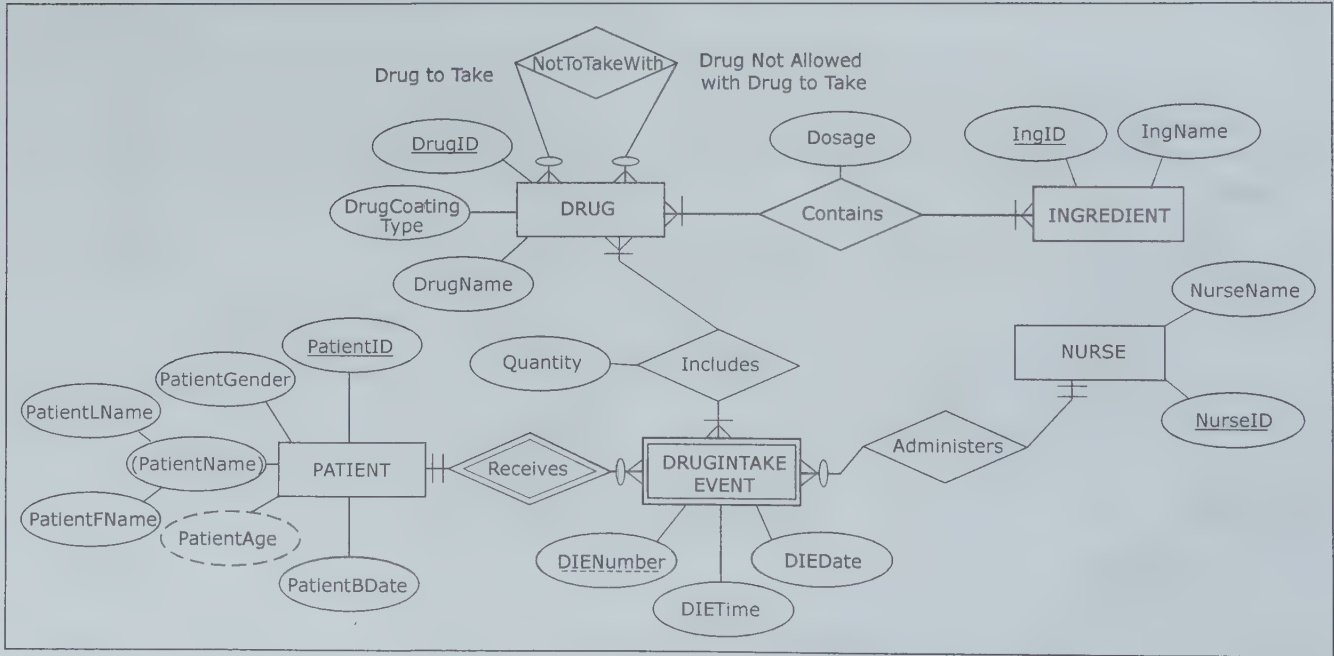


Figure 2.64 An ER diagram for the Midtown Memorial Patients Drug Dispense Database